

Path Planning- Graph search, deterministic graph search

Module 5

Introduction to Path Planning

- Path planning of a mobile robot is one of the basic operations needed to implement the navigation of the robot.
- The path planning operation provides the answer to the question “how should I get to where I” am going?”
- Path planning gives a feasible collision free path for going from one place to another.

Introduction to Path Planning

- On the basis of the way the information about the robot's environment is obtained, most of the path planning methods can be classified into two categories:

Model-based approach

- In the model based approach, all the information about the robot's workspace are pre learned, and the user specifies the geometric models of objects and a description of them in terms of these models.

Model-free approach

- In the model-free approach, some of the information about the robot's environment is obtained via sensors (e.g., vision, range, touch sensors).
- The user has to specify all the robotic motions needed to accomplish a task.

Introduction to Path Planning

- Path planning is a fundamental aspect of robotics and artificial intelligence, focusing on determining a collision-free and optimal path from a start point to a goal within a defined environment.
- One widely used method in path planning is graph search, which represents the environment as a graph where nodes correspond to discrete locations and edges represent possible transitions between them.

Graph Search Techniques

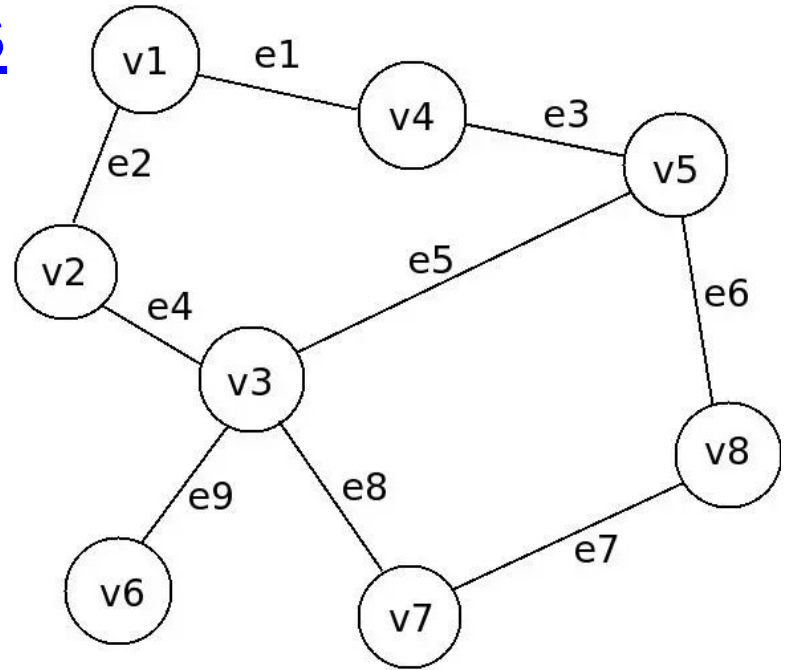
- Graph representation is a fundamental concept in **path planning**, particularly in **robotics, AI, and computer science**.
- It allows the modeling of environments as a set of nodes and edges, making it easier to compute optimal paths for navigation.

Graph Search Techniques

A graph consists of:

- **Nodes (Vertices):** Represent locations in the environment.
- **Edges (Links):** Represent possible paths between locations with associated costs (e.g., distance, time, energy).

Graph search techniques operate by exploring nodes and edges to determine an optimal path based on predefined criteria such as shortest distance or minimum



An undirected graph with 8 vertices and 9 edges

Deterministic Graph Search

- Deterministic graph search algorithms operate in a predictable manner, meaning that given the same input, they always produce the same output.
- These methods guarantee an optimal or suboptimal solution depending on the algorithm used.

1. Breadth-First Search (BFS)

- Breadth-First Search (BFS) is a fundamental graph traversal algorithm used for path planning, especially in unweighted graphs.
- It explores all neighbor nodes at the present depth level before moving on to nodes at the next depth level.
- BFS guarantees finding the shortest path in an unweighted graph.

1. Breadth-First Search (BFS)

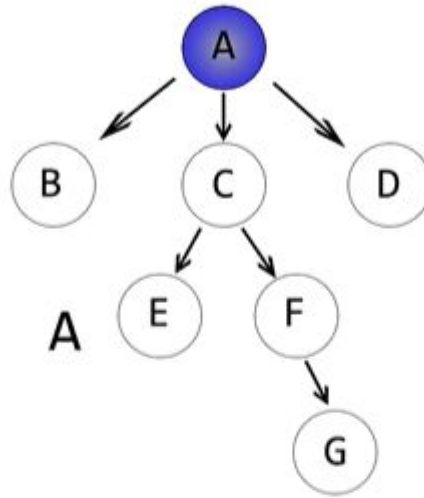
Key Characteristics

- Explores nodes level by level.
- Uses a **queue (FIFO - First In, First Out)** for node expansion.
- Guarantees the **shortest path** in an unweighted graph.
- **High memory usage** due to storing multiple nodes in the boundary.
- Suitable for **searching shortest paths** in mazes, network routing, and AI applications.

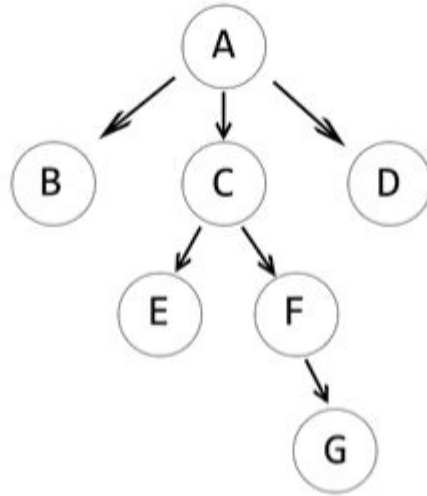
1. Breadth-First Search (BFS)

Algorithm Steps

1. Initialize a queue and enqueue the start node.
2. Mark the start node as visited.
3. While the queue is not empty:
 - Dequeue a node.
 - Process the node (check if it is the goal).
 - Enqueue all unvisited neighboring nodes and mark them as visited.
4. Repeat until the goal is found or the queue is empty.



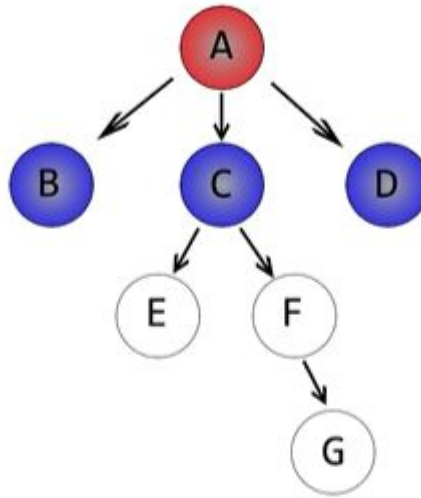
Queue



Queue

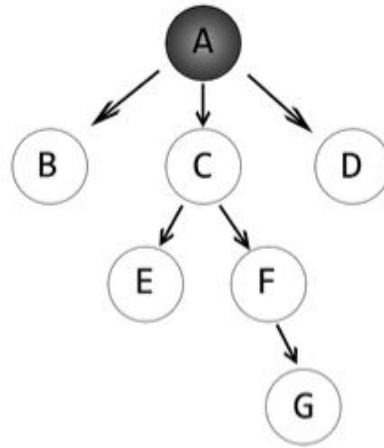
A





Queue

A



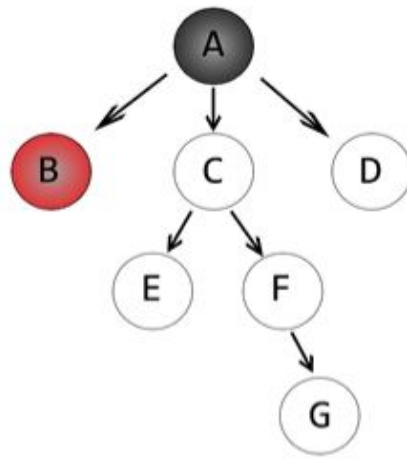
Queue

B C D

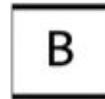


Processed

A

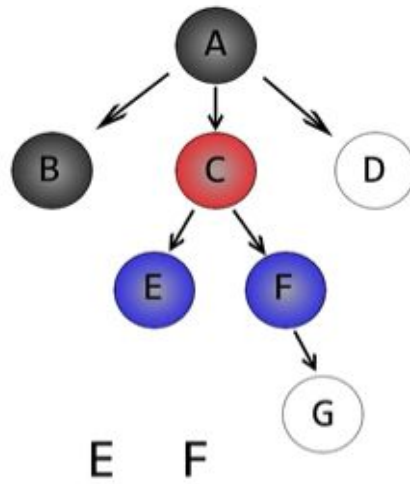


Queue



Processed



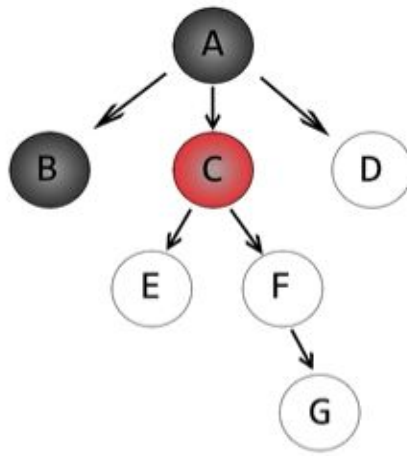


Queue



Processed





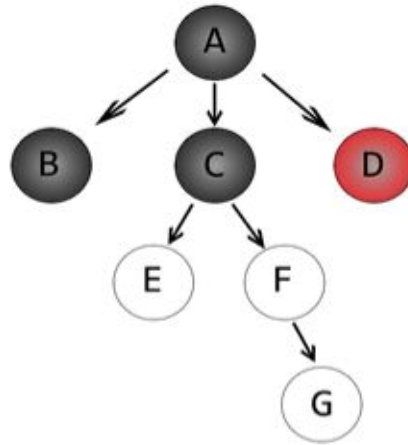
Queue

D E F

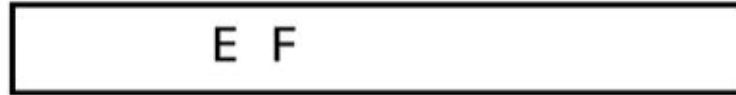
C

Processed

AB

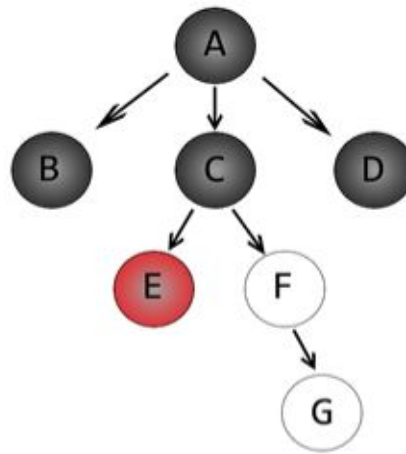


Queue

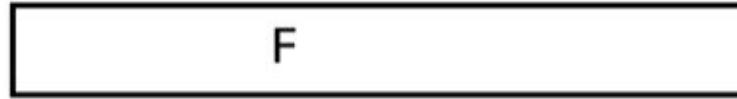


Processed



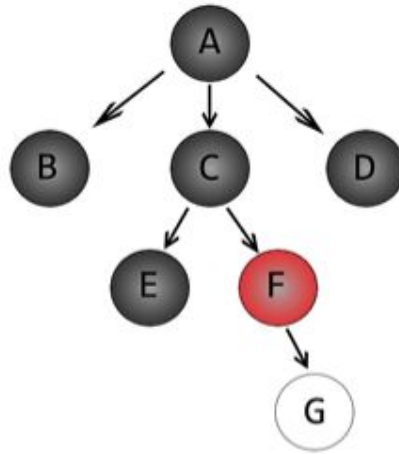


Queue



Processed



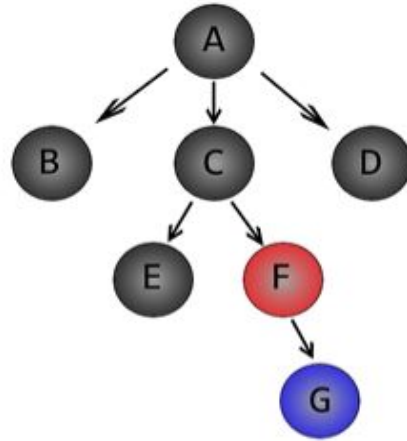


Queue

F

Processed

ABCDE



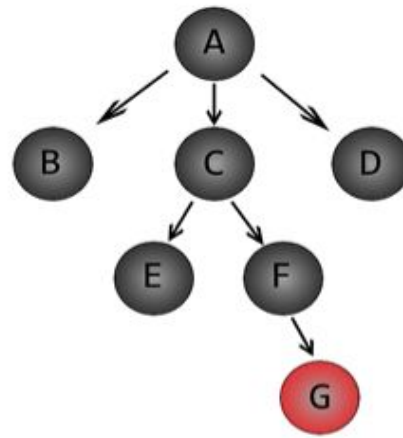
Queue

G

F

Processed

ABCDE

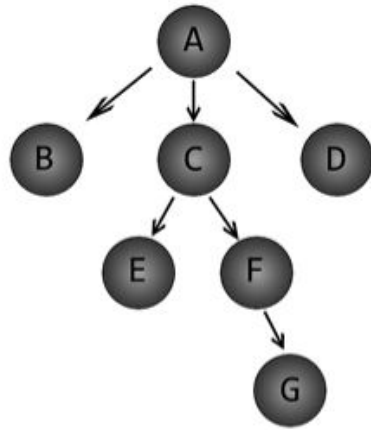


Queue

G

Processed

ABCDEF



Queue

☐

Processed

ABCDEFGG

2.Depth-First Search (DFS)

- Depth-First Search (DFS) is a graph traversal algorithm that explores as far down a branch as possible before backtracking.
- It is commonly used in applications like maze solving, cycle detection, and topological sorting.

Depth-First Search (DFS)

Key Characteristics

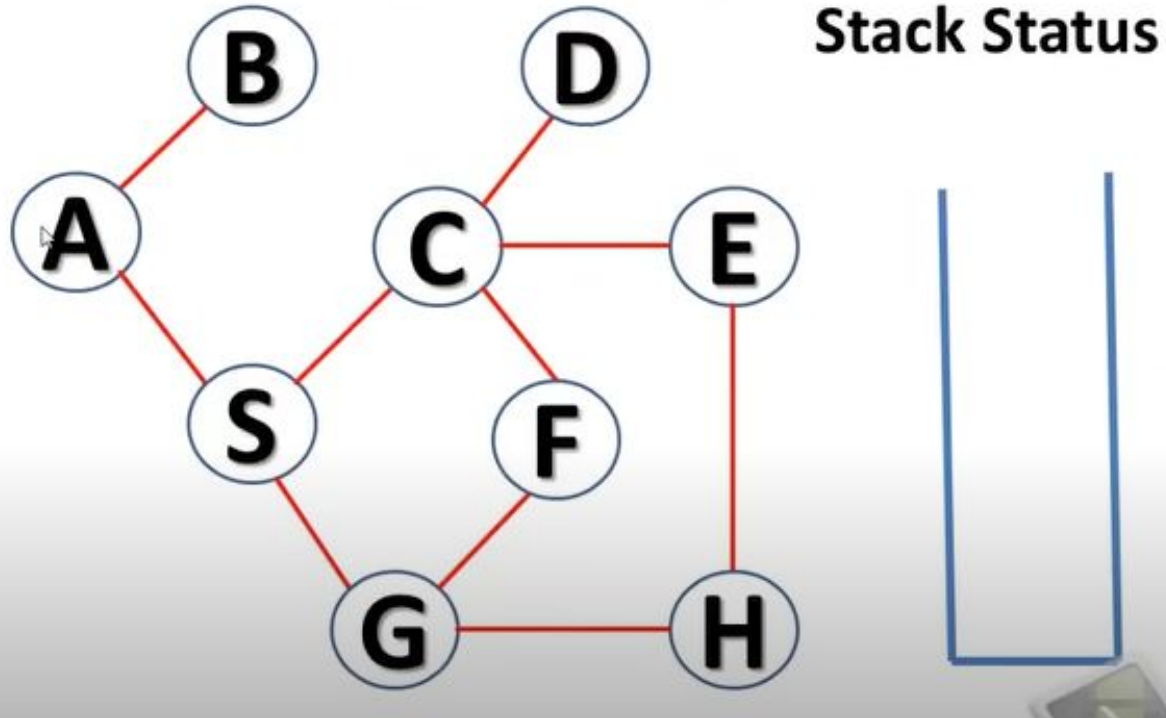
- Explores nodes deeply before backtracking.
- Uses a **stack (LIFO - Last In, First Out)** for node expansion.
- Does **not guarantee the shortest path**.
- More memory-efficient than BFS for deep graphs.
- Can get stuck in infinite loops in cyclic graphs unless visited nodes are tracked.

DFS vs. BFS (Breadth-First Search)

Feature	DFS (Depth-First Search)	BFS (Breadth-First Search)
Traversal Approach	Explores deep first	Explores level by level
Data Structure Used	Stack (or Recursion)	Queue
Memory Usage	$O(V)$ (lower for sparse graphs)	$O(V)$ (higher in wide graphs)
Path Finding	Not optimal	Finds shortest path (unweighted graphs)
Cycle Detection	Yes	Yes
Applications	Solving puzzles, Topological sorting, Connected components	Shortest path, Network broadcasting, GPS Navigation

DEPTH FIRST SEARCH

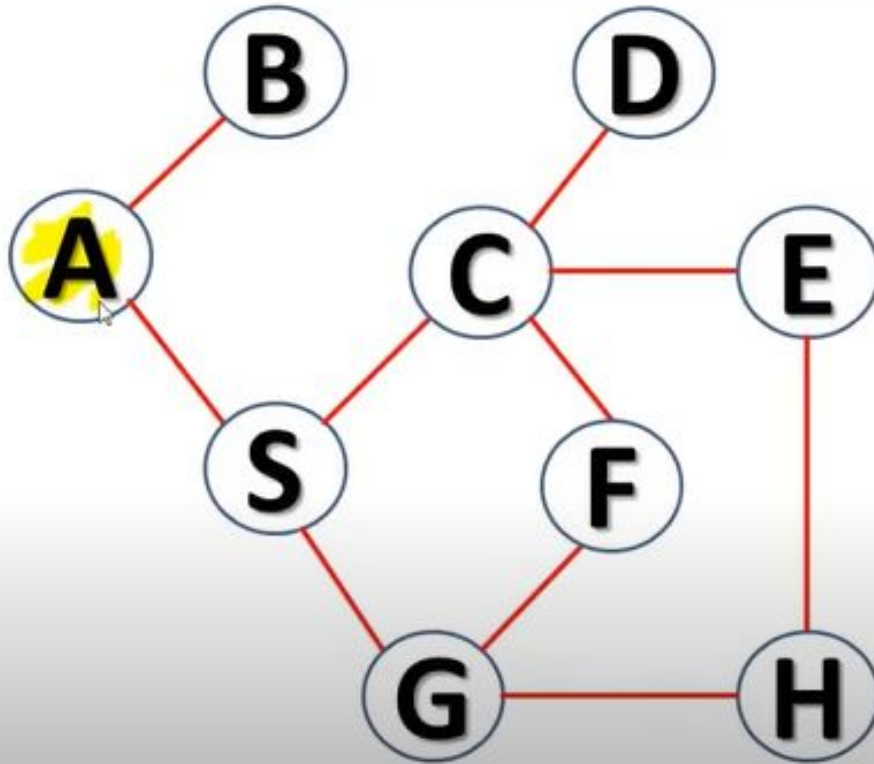
Stack Status



- Empty Stack -used for keeping track of visited nodes
- Start With Node A

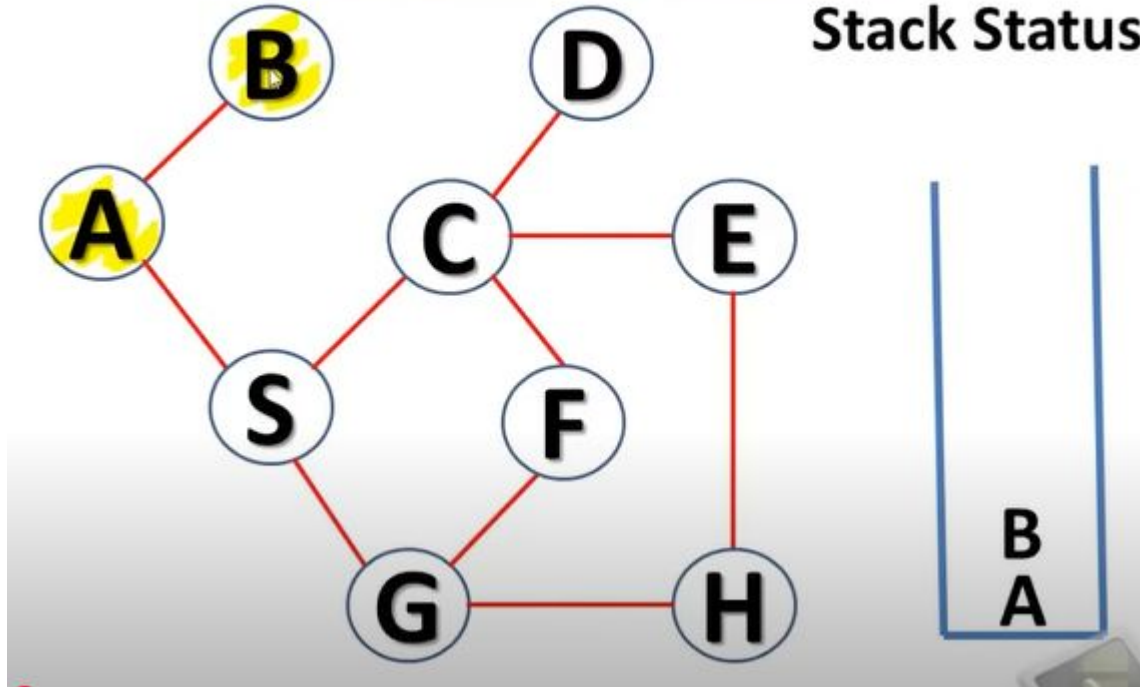
DEPTH FIRST SEARCH

Stack Status



Output : **A**

DEPTH FIRST SEARCH

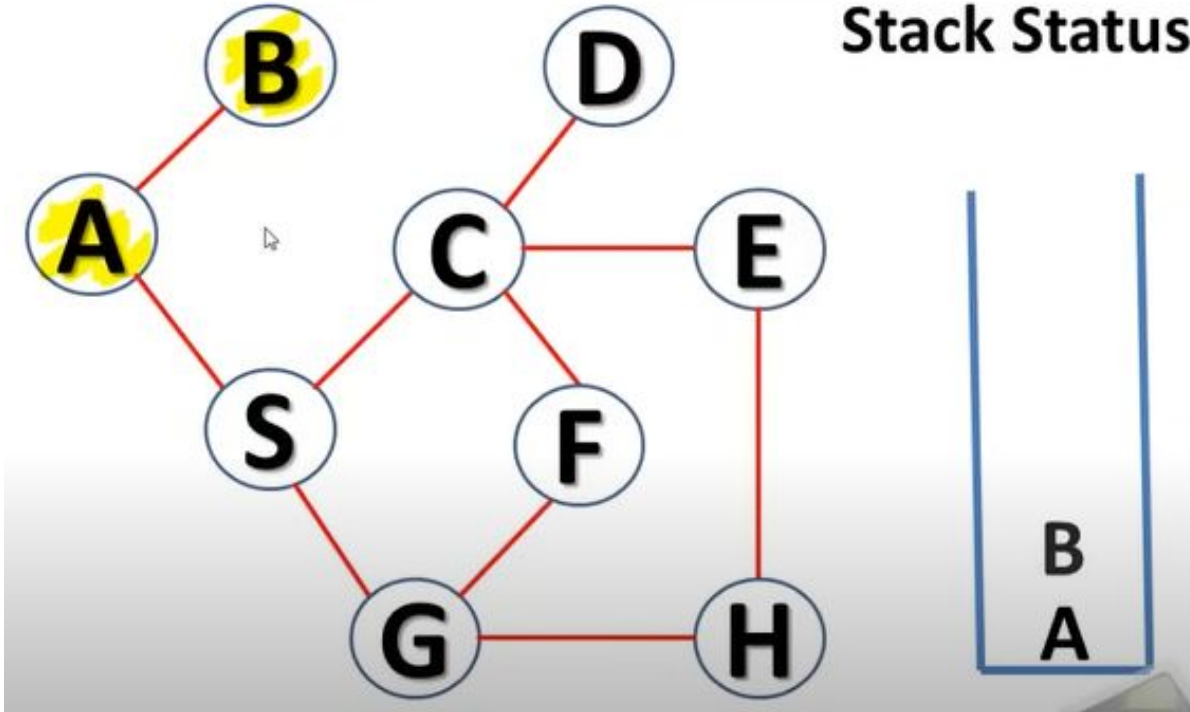


- Take Alphabetical order.

Output : **AB**

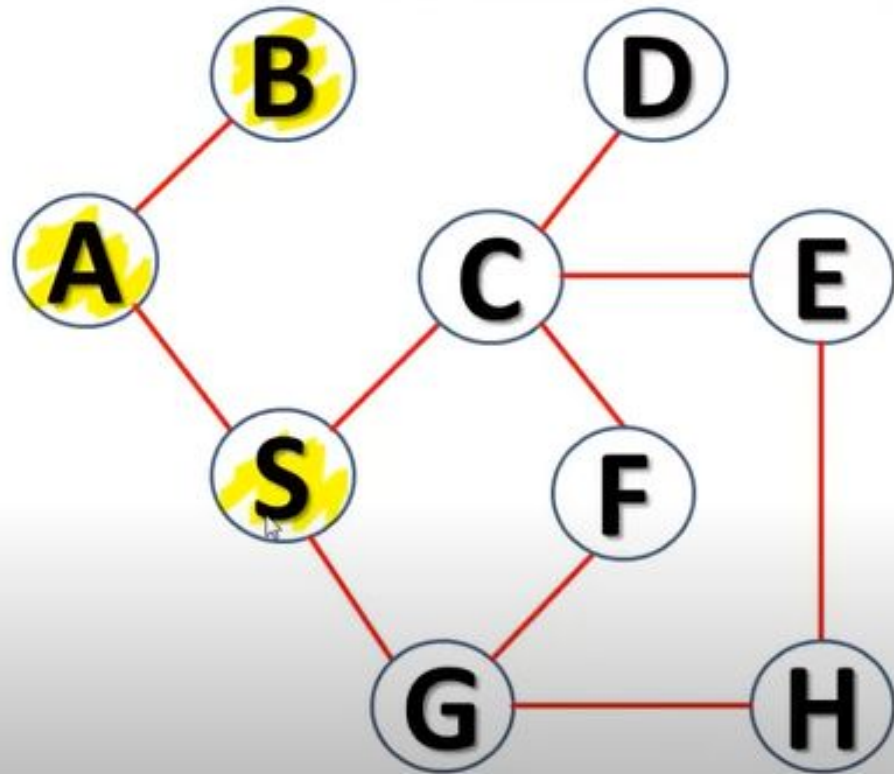
DEPTH FIRST SEARCH

Stack Status



- A is only adjacent node for B
- So there is no other go
- So B will remove from the Stack.

DEPTH FIRST SEARCH



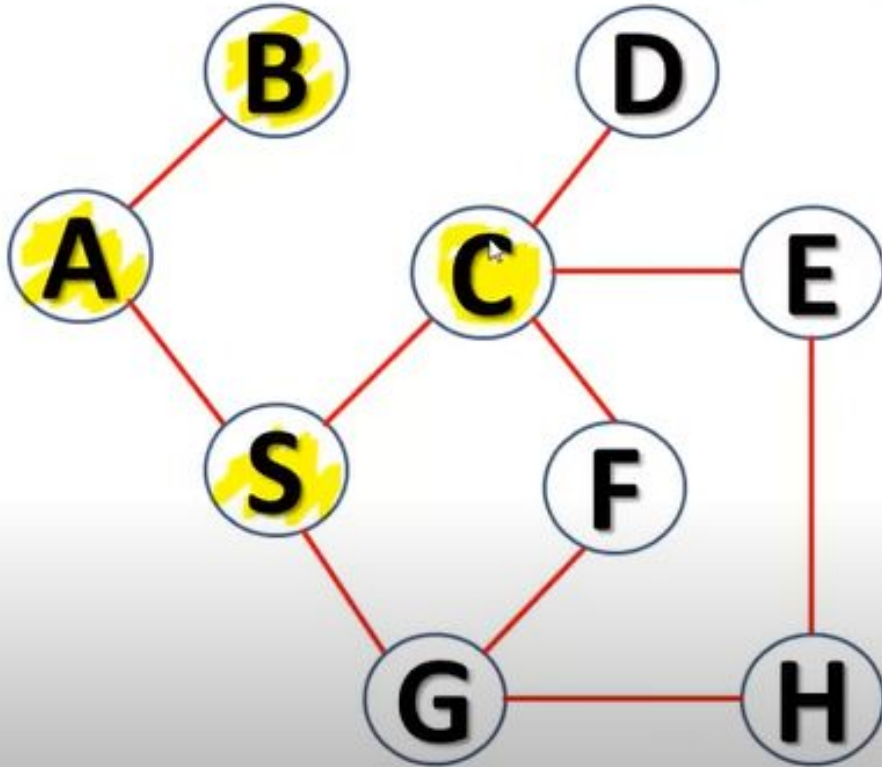
Stack Status



Output : **ABS**

DEPTH FIRST SEARCH

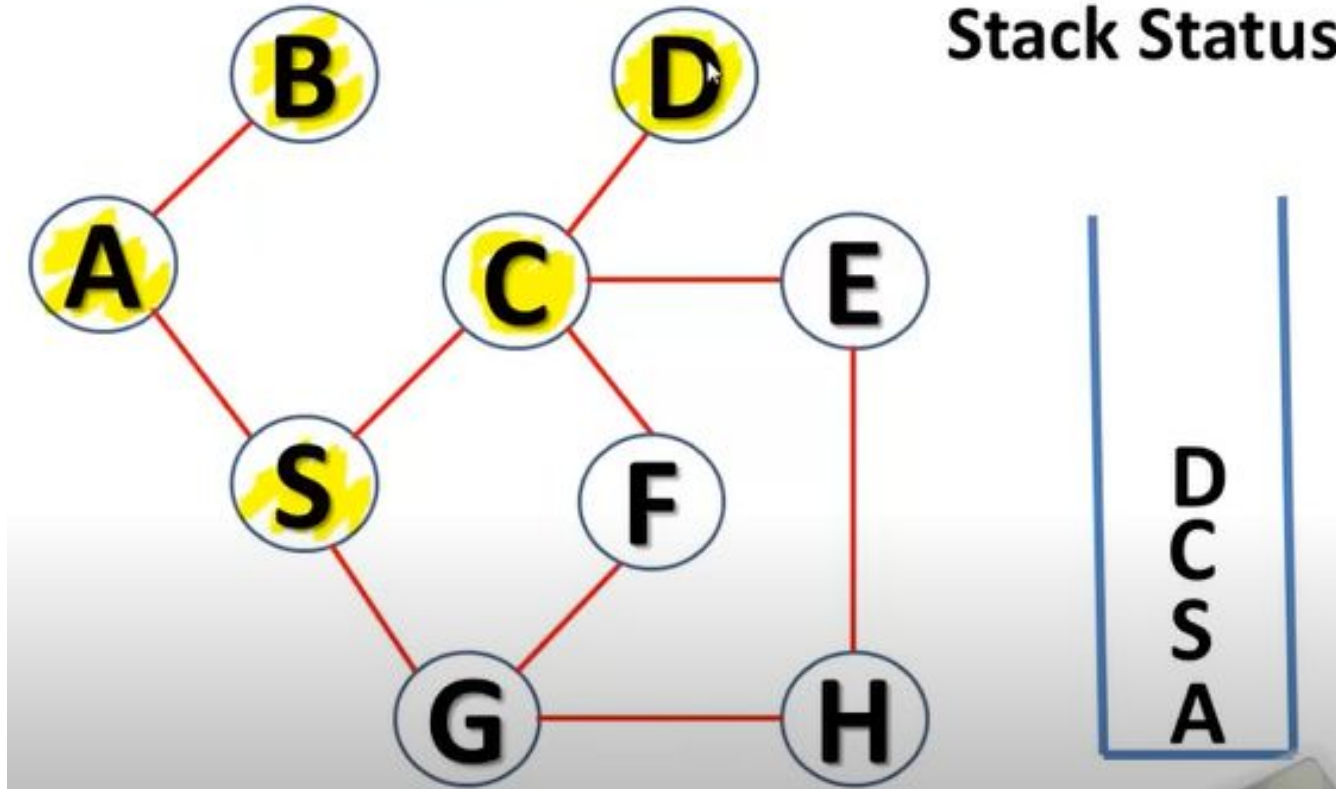
Stack Status



Output : **ABSC**

DEPTH FIRST SEARCH

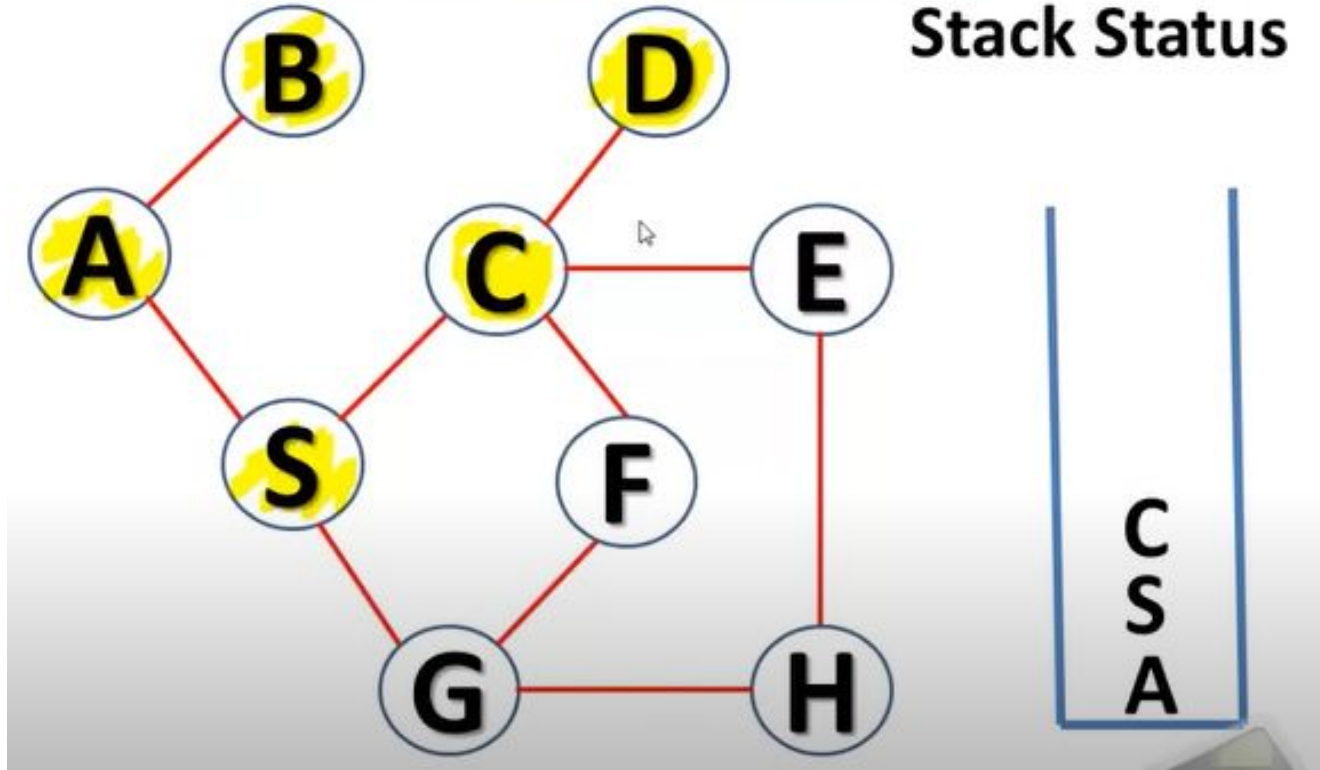
Stack Status



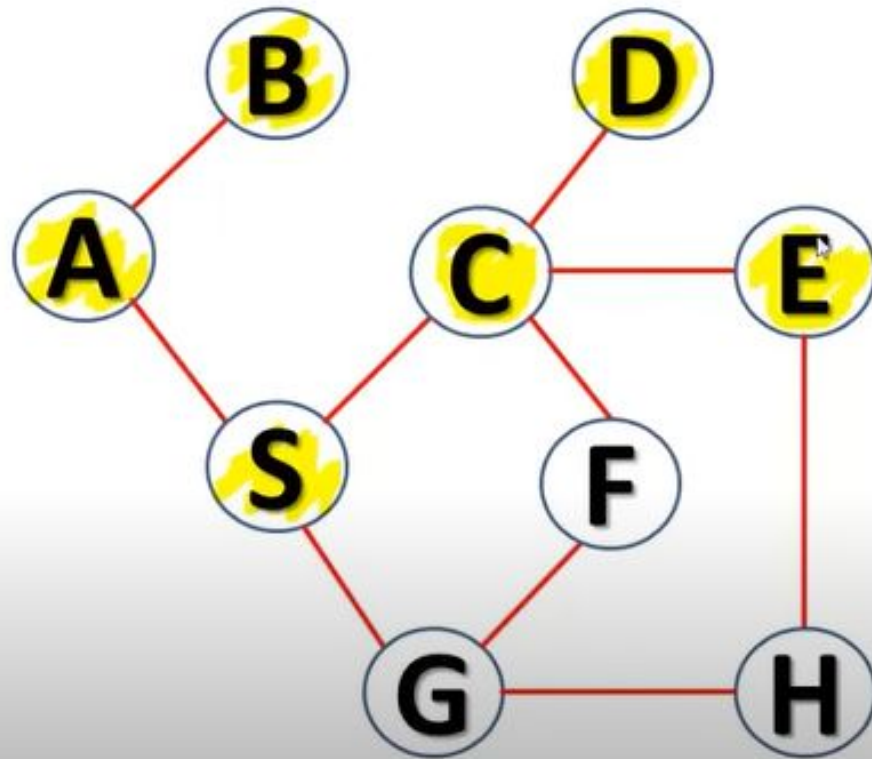
Output : **ABSCD**

DEPTH FIRST SEARCH

Stack Status



DEPTH FIRST SEARCH



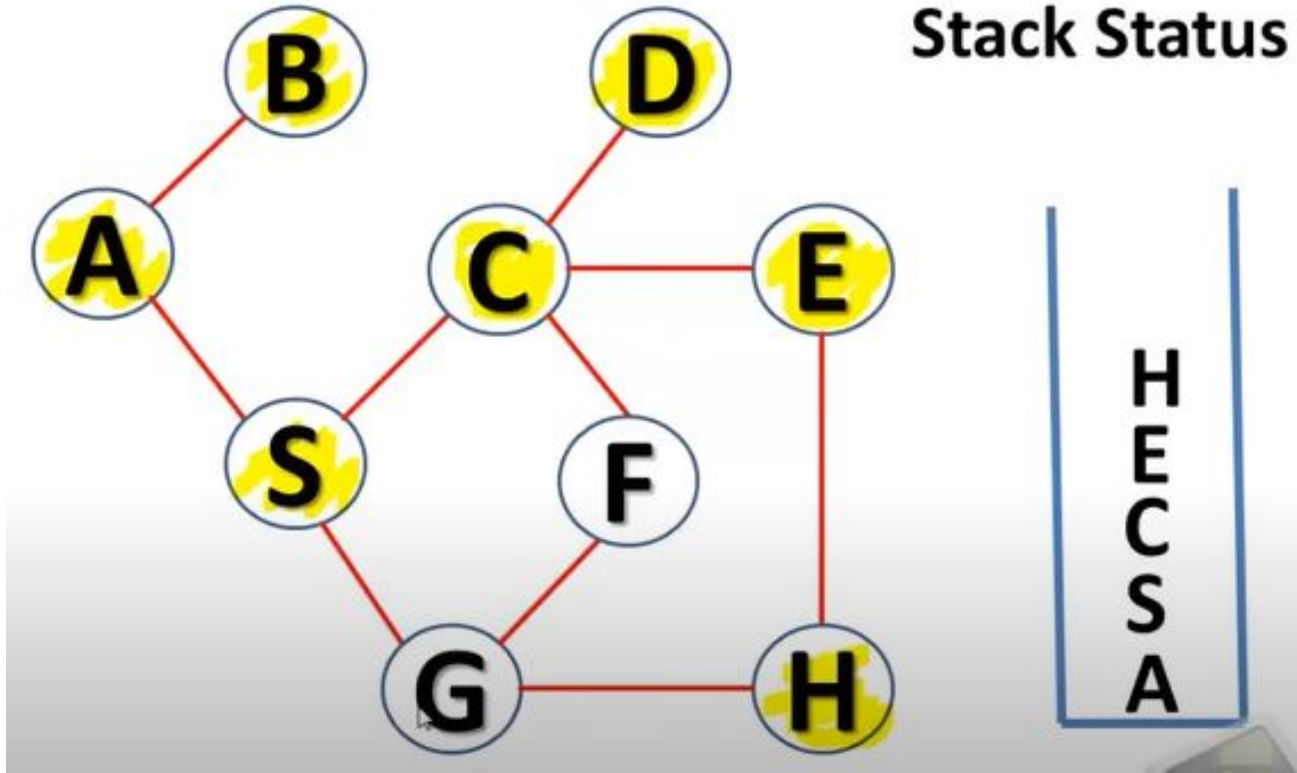
Stack Status

E
C
S
A

Output : **ABSCDE**

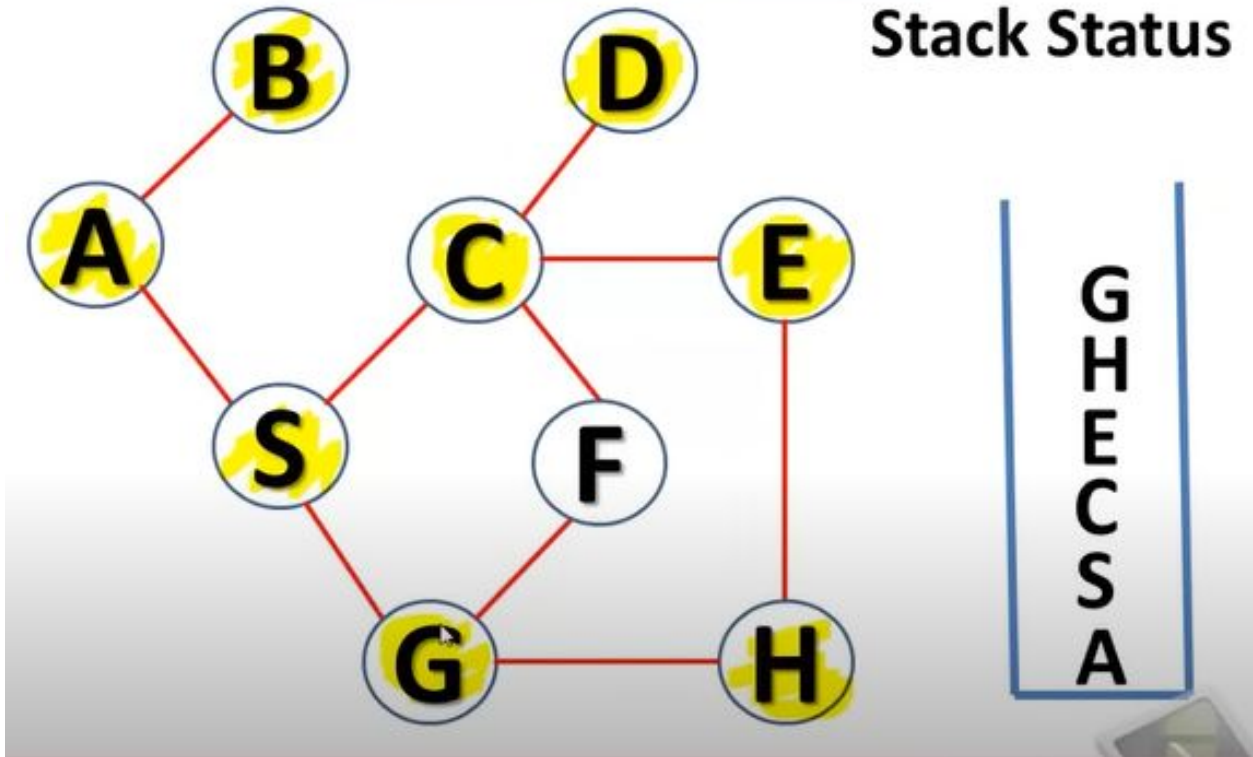
DEPTH FIRST SEARCH

Stack Status



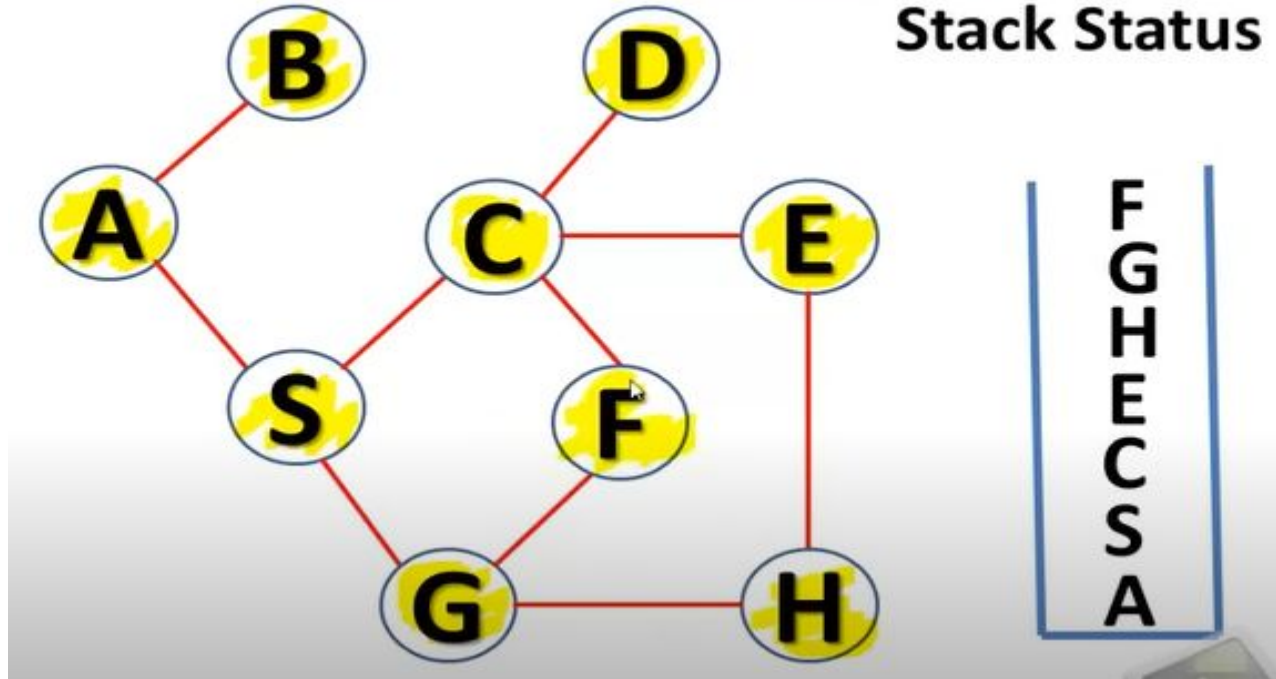
Output : **ABSCDEH**

DEPTH FIRST SEARCH



Output : **ABSCDEHG**

DEPTH FIRST SEARCH



Output : **ABSCDEHGF**

Dijkstra's Algorithm

- Dijkstra's Algorithm is a **greedy algorithm** used to find the **shortest path** from a **single source node** to all other nodes in a **weighted graph**.
- It works with both **directed** and **undirected graphs**, as long as all edge weights are **non-negative**.

Dijkstra's Algorithm

Characteristics of Dijkstra's Algorithm

- Uses a priority queue (min-heap) for efficiency.
- Guaranteed to find the shortest path in graphs with non-negative weights.
- Time complexity depends on the implementation:
 - Using a simple array: $O(V^2)$
 - Using a binary heap (priority queue): $O((V + E) \log V)$
- Commonly used for shortest path problems in road networks, network routing, and AI navigation.

Dijkstra's Algorithm

Algorithm Steps

1. Initialization:

- Set the **source node's distance to 0** and all others to infinity.
- Mark all nodes as **unvisited**.
- Use a **priority queue** (min-heap) to keep track of nodes with the smallest known distance.

2. Relaxation Process:

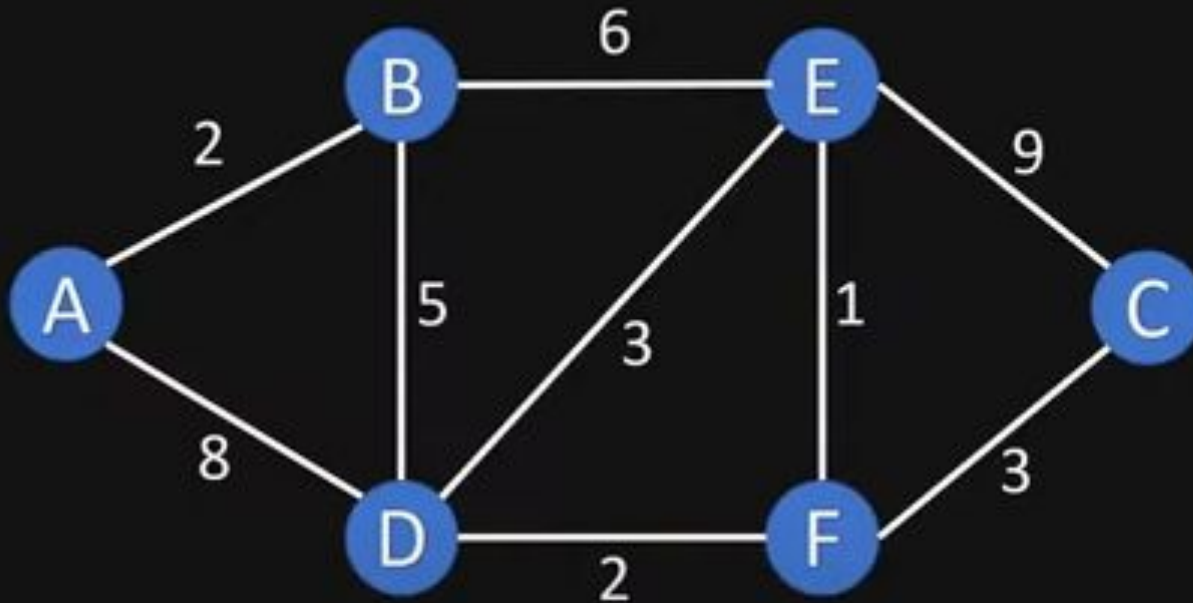
- Extract the **node with the smallest distance** from the priority queue.
- Update the distances of its **adjacent nodes** if a shorter path is found.
- Continue until all nodes have been visited.

Dijkstra's Algorithm

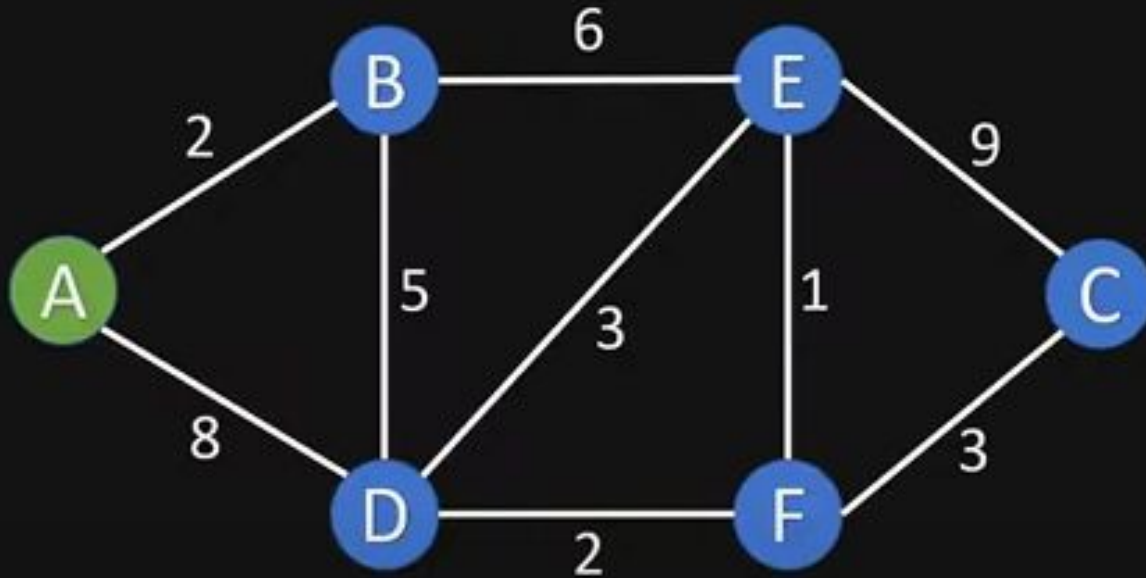
3. Termination:

- Once the destination node is extracted from the priority queue, the shortest path is found.
- If all reachable nodes have been processed, the algorithm stops.

Dijkstra's Shortest Path Algorithm



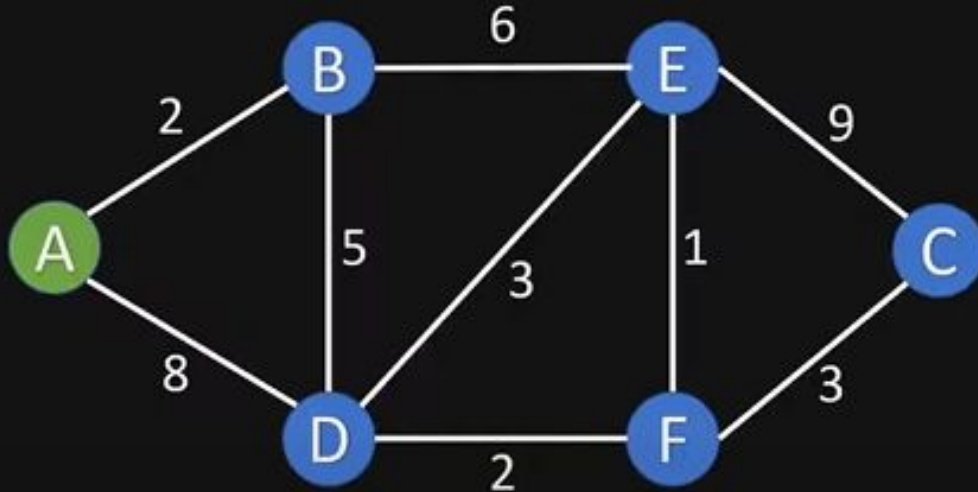
1. Mark all nodes as unvisited



Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

2. Assign to all nodes a tentative distance value

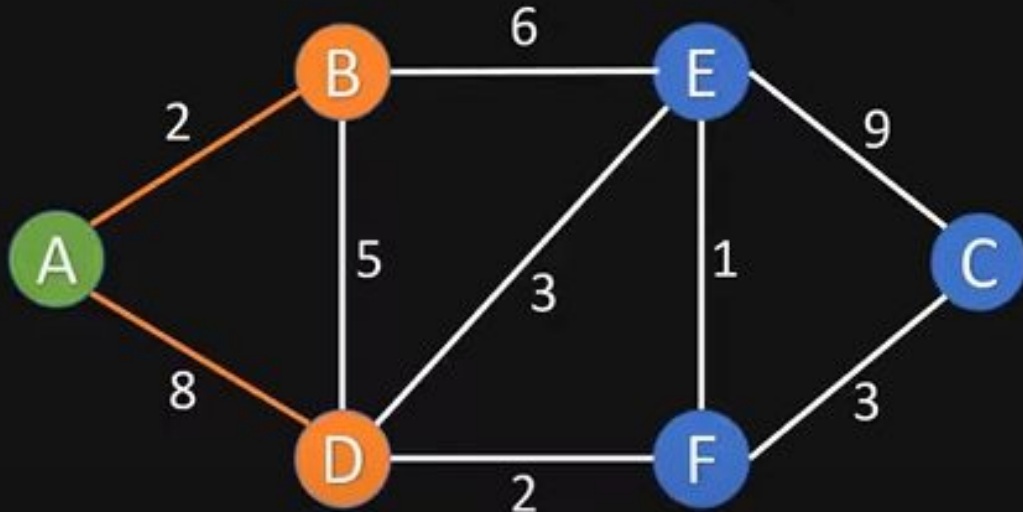


Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	∞	
C	∞	
D	∞	
E	∞	
F	∞	

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance

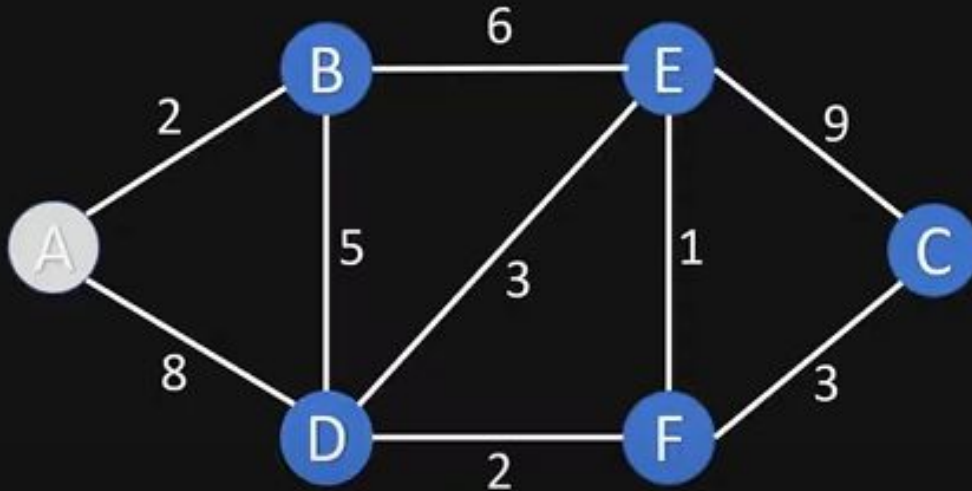


Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

4. Mark current node as visited

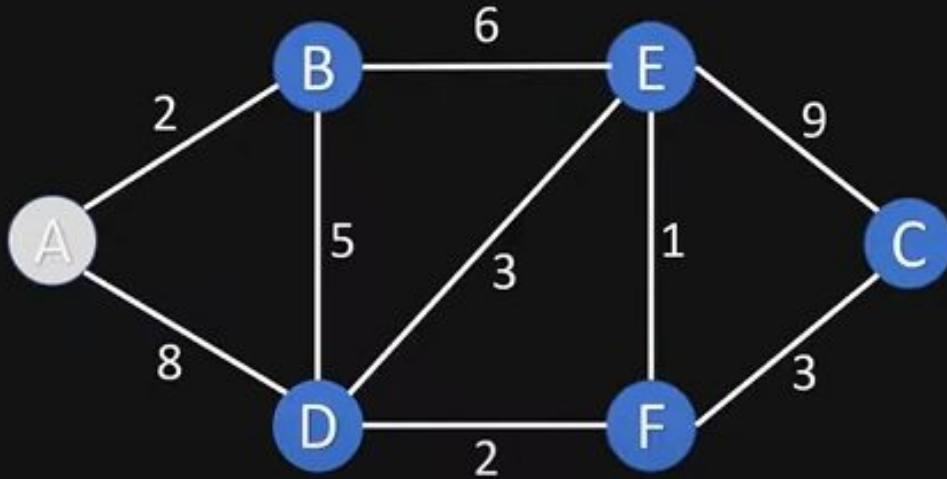


Visited Nodes: []

Unvisited Nodes: [A, B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

5. Choose new current node from unvisited nodes with minimal distance



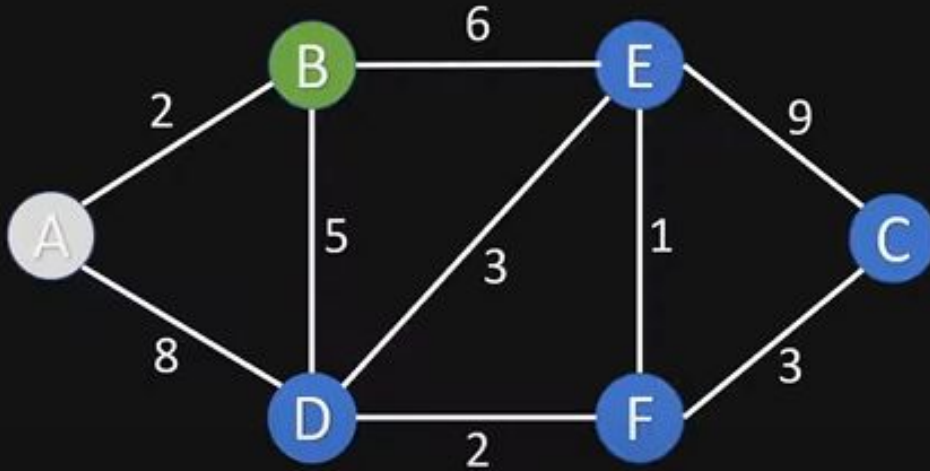
Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

5. Choose new current node from unvisited nodes with minimal distance

- Current unvisited node is B



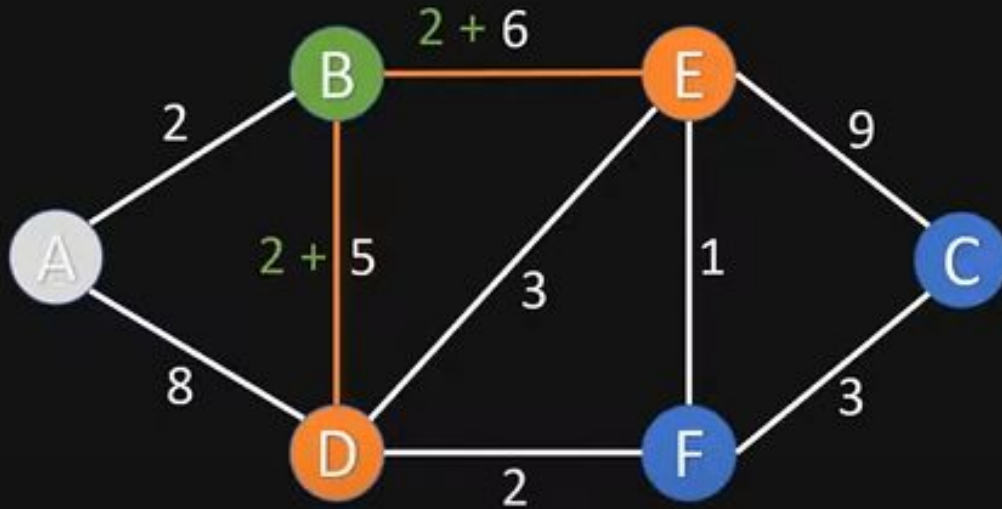
Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

- Go to step 3

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

3. For the current node calculate the distance to all unvisited neighbours

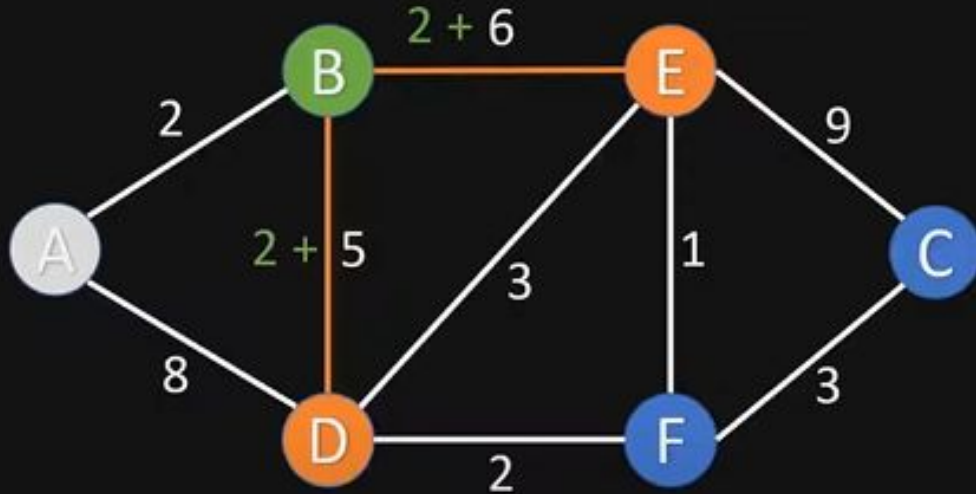


Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	8	A
E	∞	
F	∞	

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance

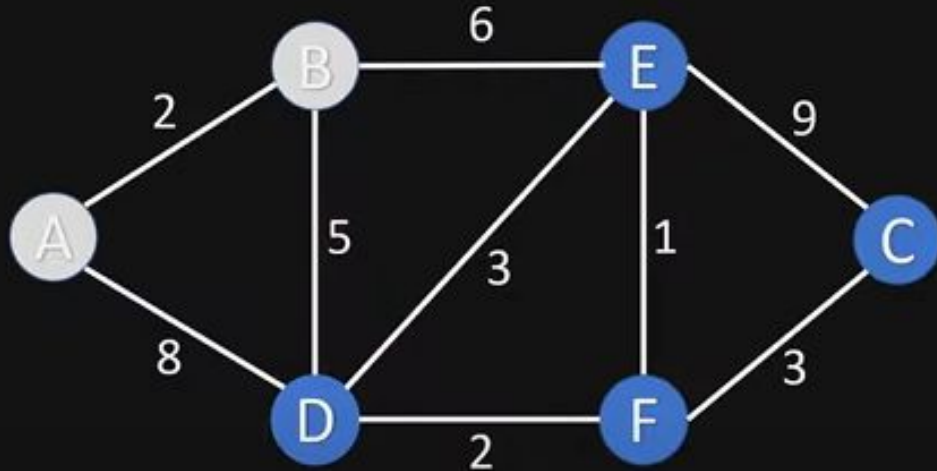


Visited Nodes: [A]

Unvisited Nodes: [B, C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

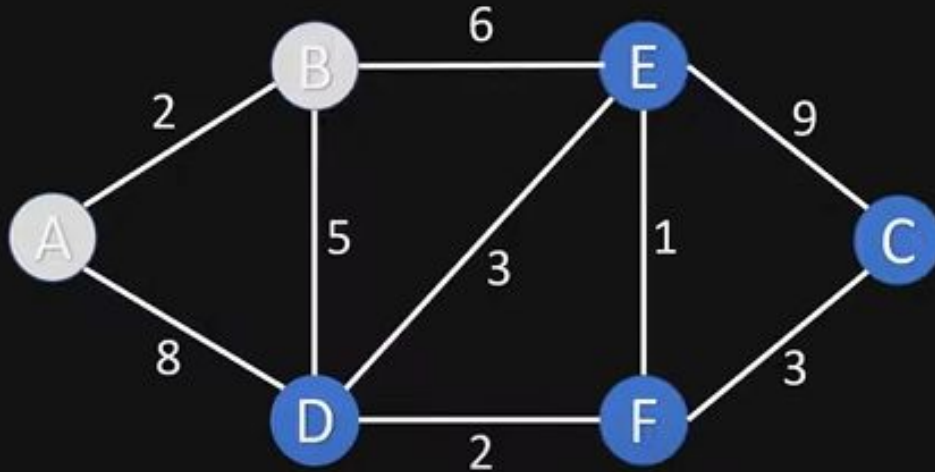
4. Mark current node as visited



Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

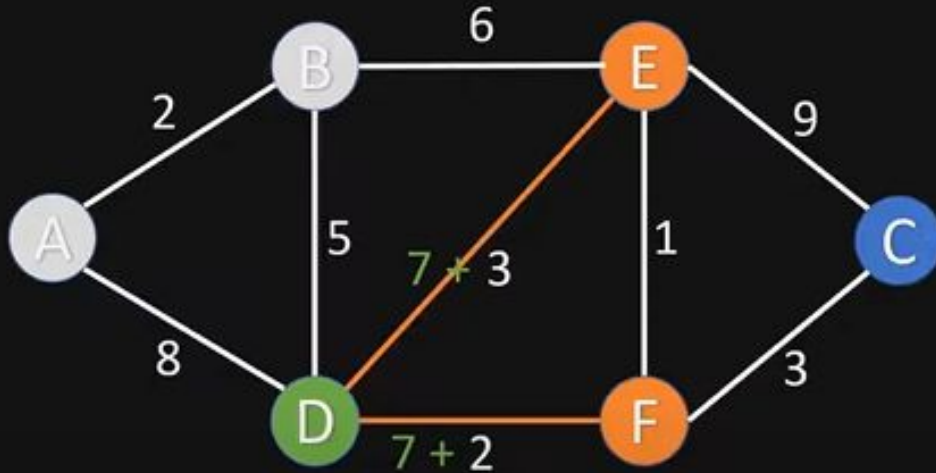
5. Choose new current node from unvisited nodes with minimal distance



Visited Nodes: [A, B] Unvisited Nodes: [C, D, E, F]

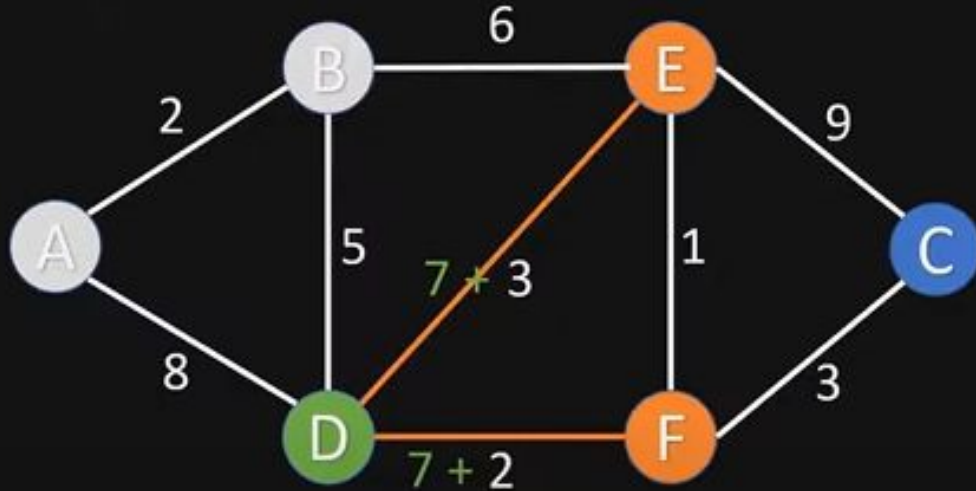
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

3. For the current node calculate the distance to all unvisited neighbours



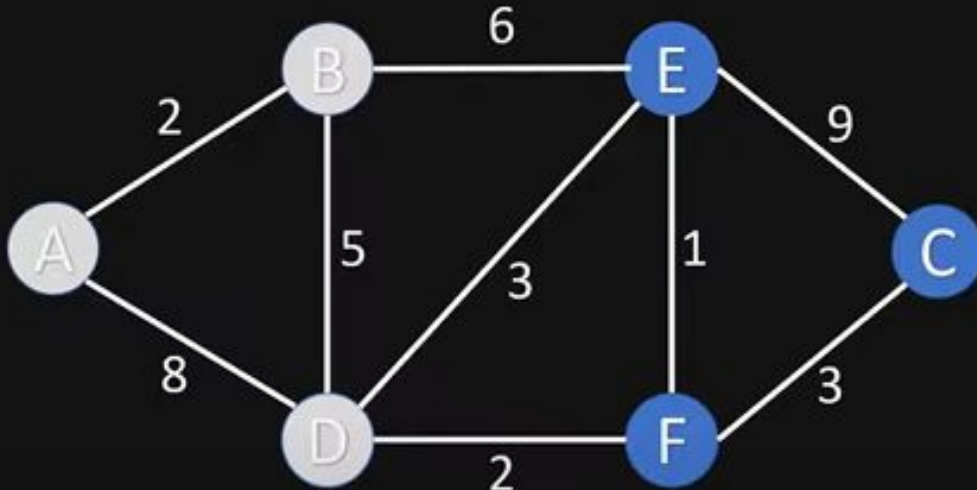
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	∞	

3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

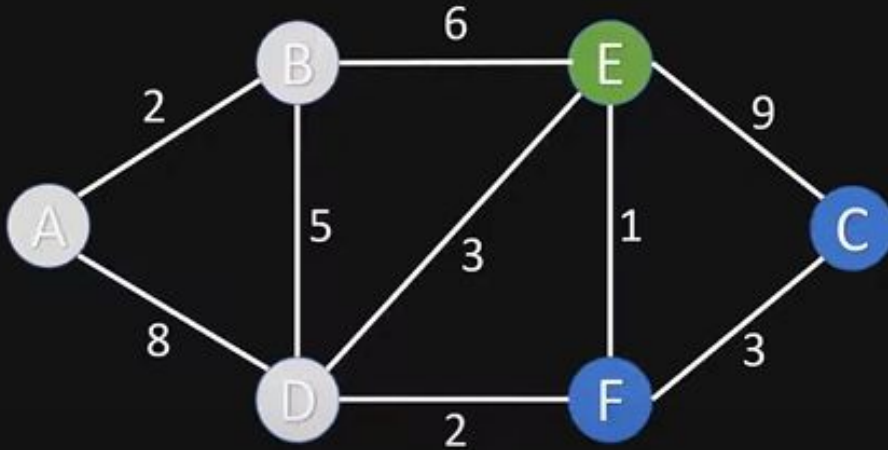
4. Mark current node as visited



Visited Nodes: [A, B, D] Unvisited Nodes: [C, D, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

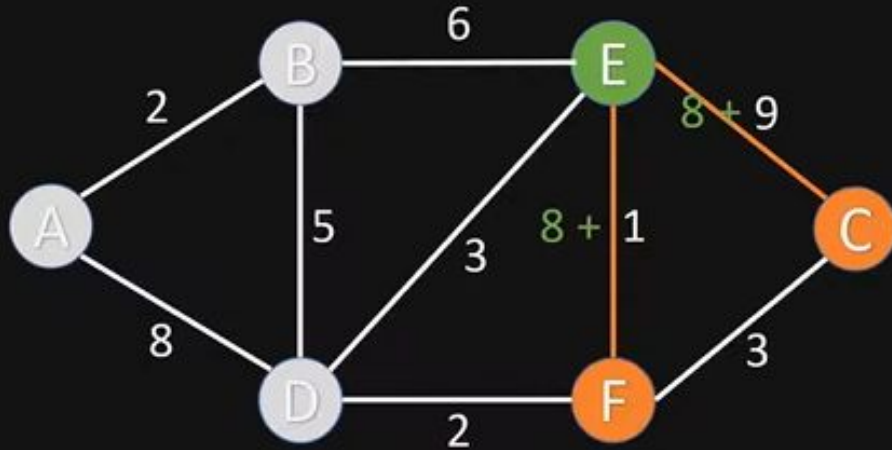
5. Choose new current node from unvisited nodes with minimal distance



Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

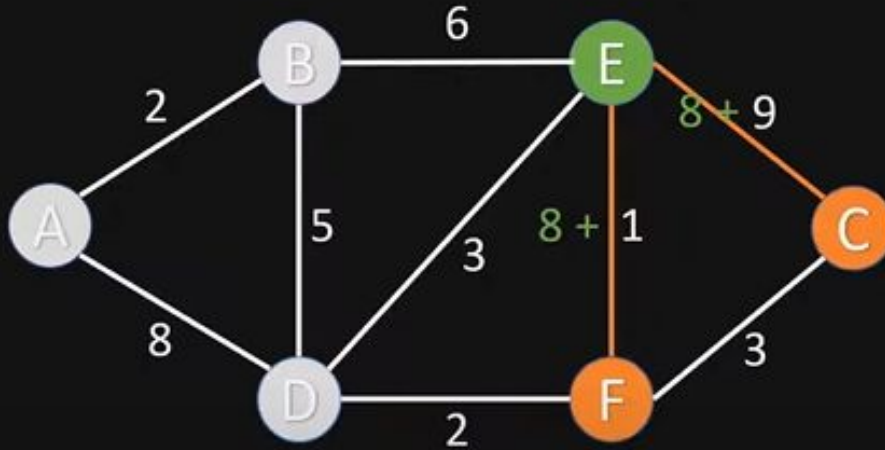
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

3. For the current node calculate the distance to all unvisited neighbours



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	∞	
D	7	B
E	8	B
F	9	D

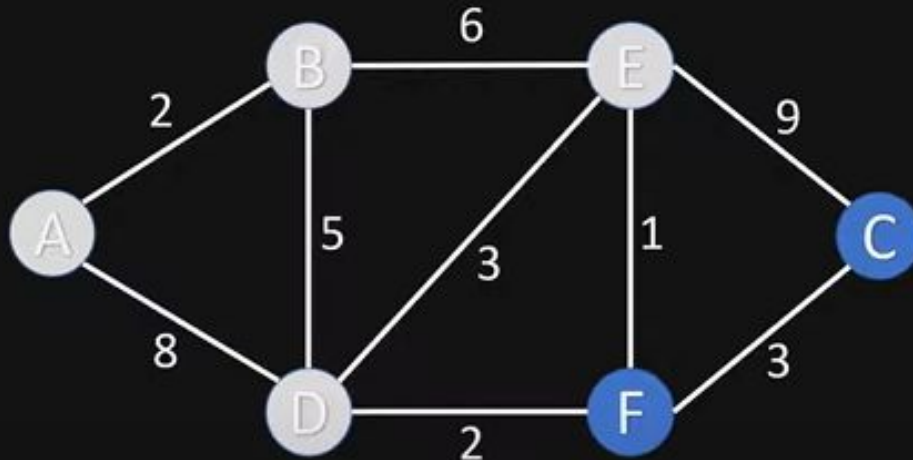
3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Visited Nodes: [A, B, D] Unvisited Nodes: [C, E, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

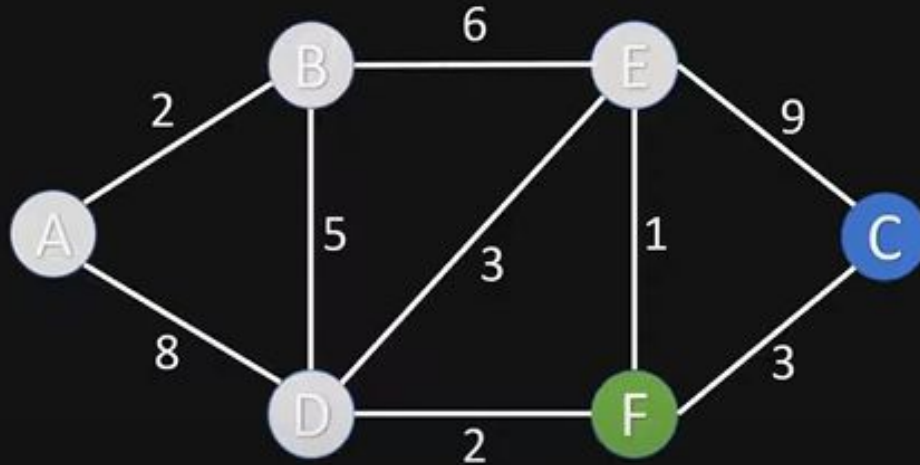
4. Mark current node as visited



Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

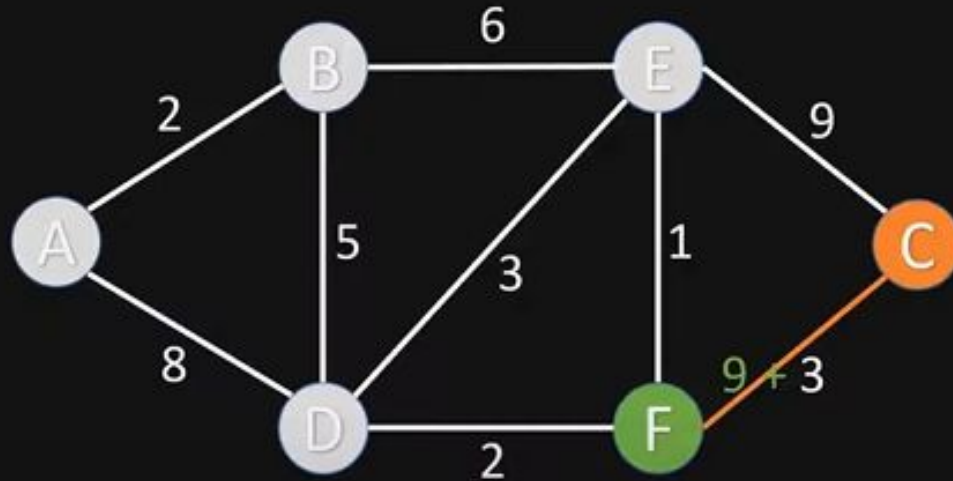
5. Choose new current node from unvisited nodes with minimal distance



Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	17	E
D	7	B
E	8	B
F	9	D

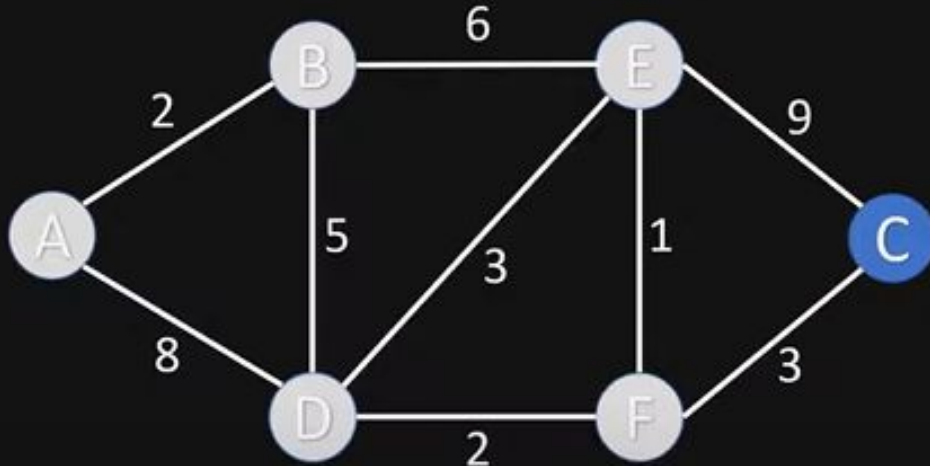
3. For the current node calculate the distance to all unvisited neighbours
3.1. Update shortest distance, if new distance is shorter than old distance



Visited Nodes: [A, B, D, E] Unvisited Nodes: [C, F]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

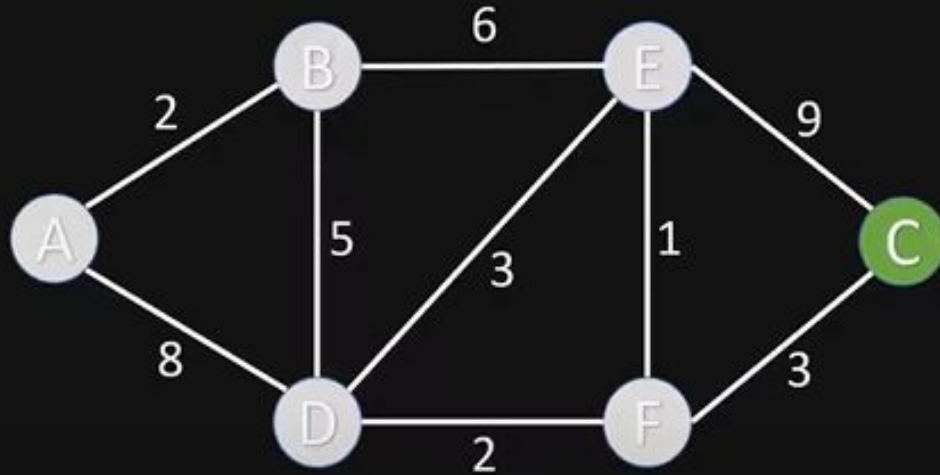
4. Mark current node as visited



Visited Nodes: [A, B, D, E, F] Unvisited Nodes: [C]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

5. Choose new current node from unvisited nodes with minimal distance

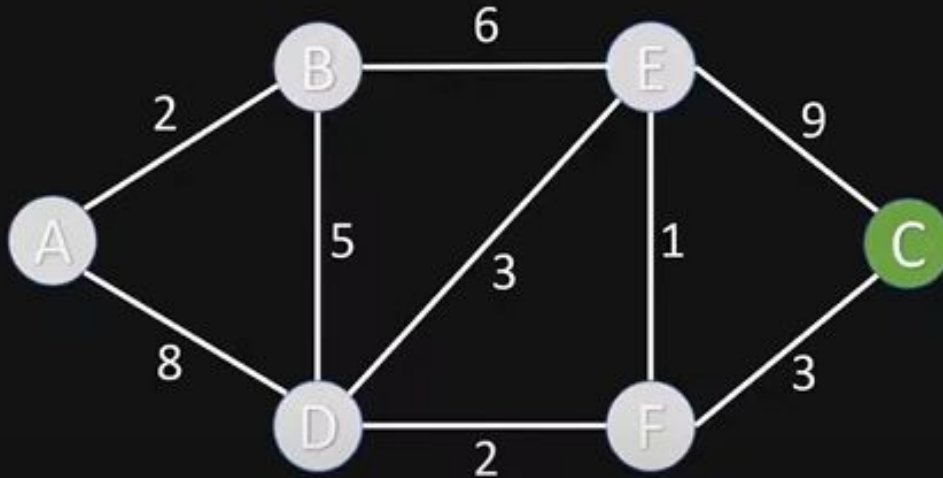


Visited Nodes: [A, B, D, E, F]

Unvisited Nodes: [C]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

3. For the current node calculate the distance to all unvisited neighbours

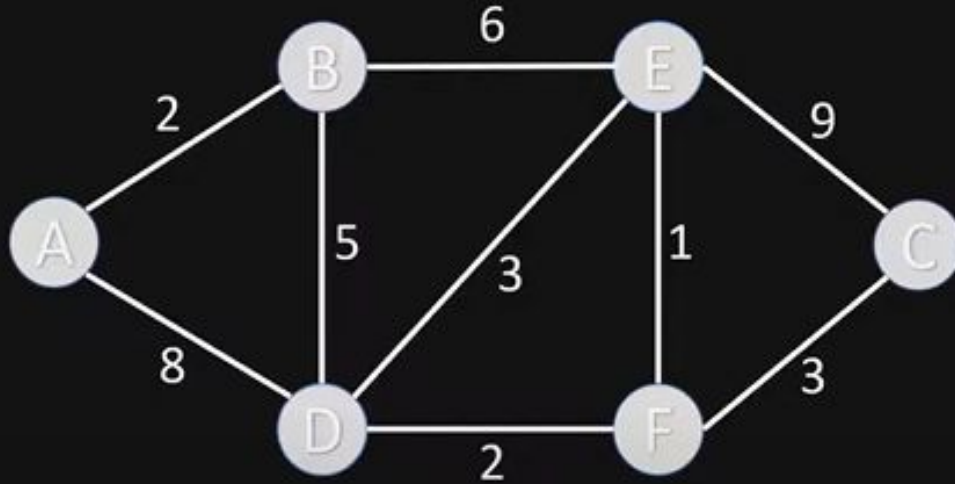


Visited Nodes: [A, B, D, E, F]

Unvisited Nodes: [C]

Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

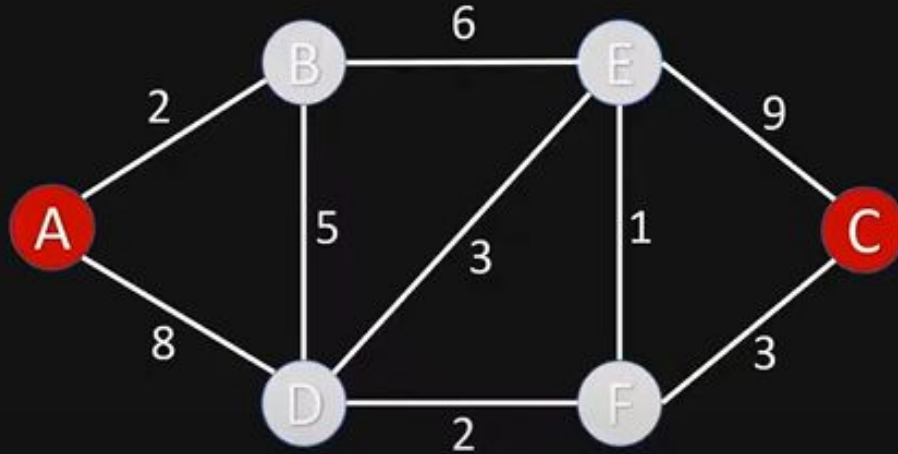
4. Mark current node as visited



Visited Nodes: [A, B, D, E, F, C] Unvisited Nodes: []

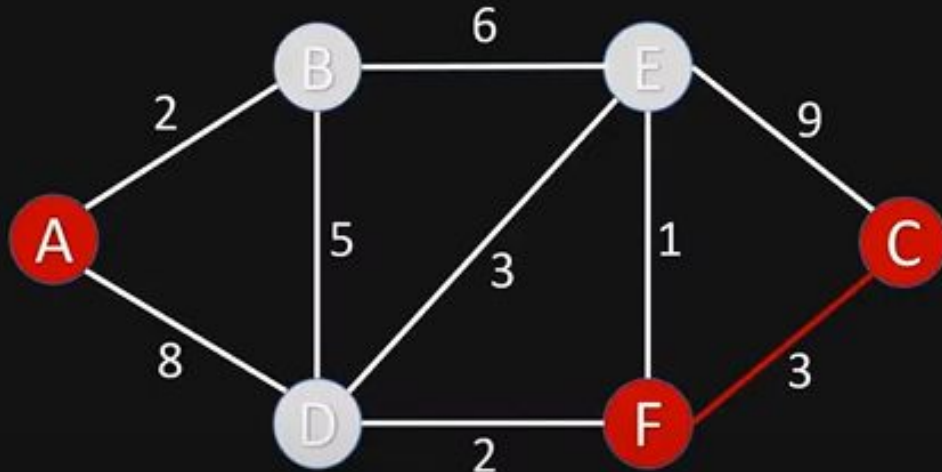
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



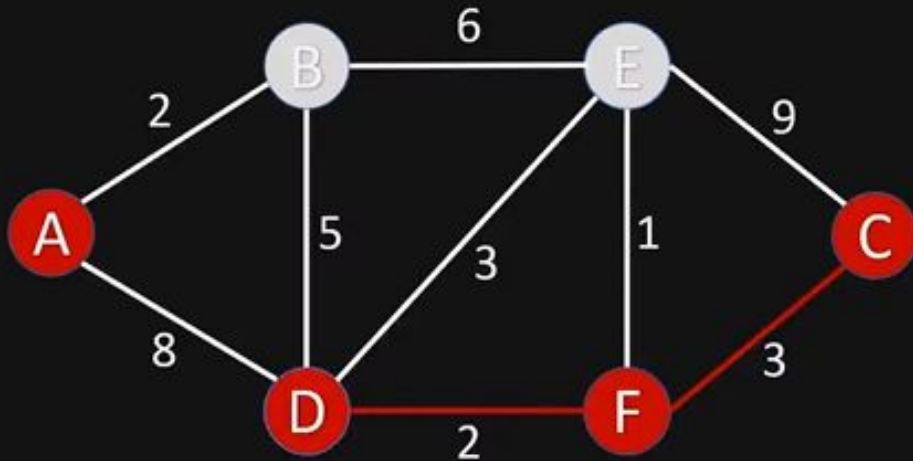
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



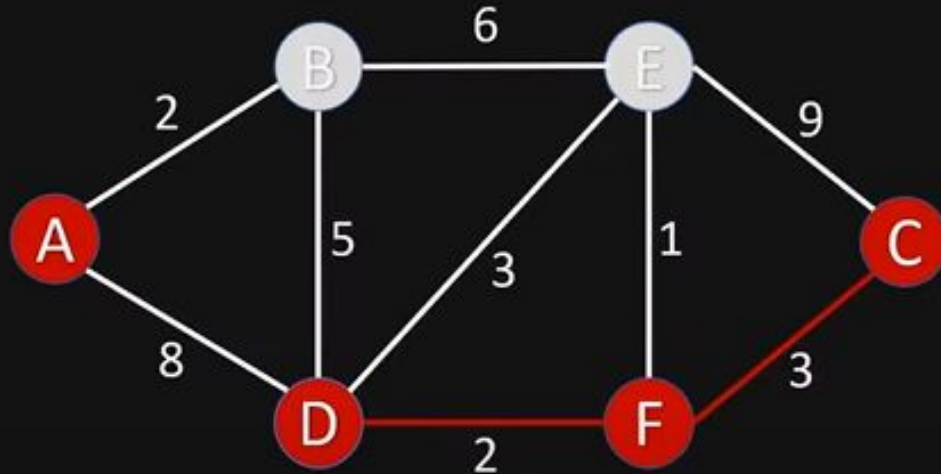
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



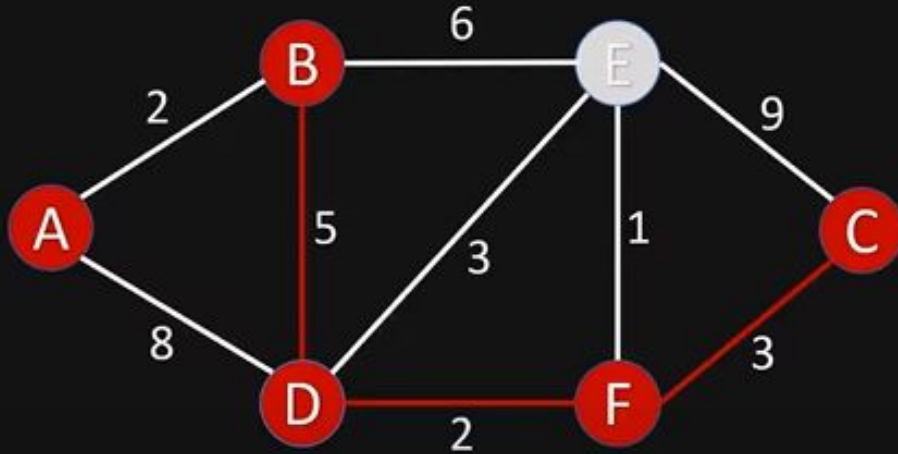
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



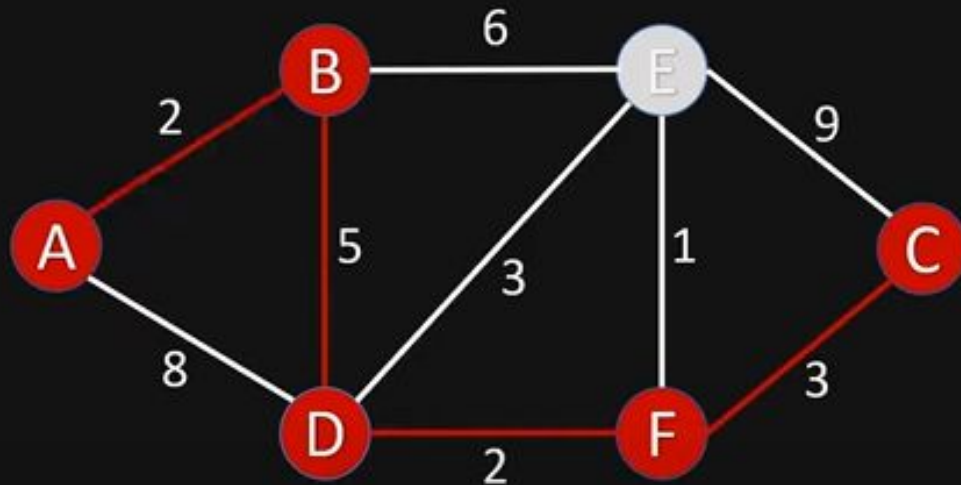
Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

Get shortest path from A to C



Node	Shortest Distance	Previous Node
A	0	
B	2	A
C	12	F
D	7	B
E	8	B
F	9	D

A-Star Algorithm

- A* (A-Star) is an **informed search algorithm** used for finding the **shortest path** in a **graph or grid**.
- It is widely used in **AI, robotics, game development, and pathfinding applications**.
- A* improves upon **Dijkstra's Algorithm** by incorporating a heuristic function, which helps guide the search towards the goal efficiently.

A-Star Algorithm

Characteristics of A Algorithm*

- **Uses both actual cost and estimated cost (heuristic).**
- **Optimized for shortest path search in weighted graphs.**
- **Works well with real-world applications (e.g., GPS navigation, AI).**
- **More efficient than Dijkstra's Algorithm in large search spaces.**

A-Star Algorithm

Cost Functions Used in A Algorithm*

A* uses two cost functions:

1. **$g(n)$** → The actual cost from the start node to the current node **n** .
2. **$h(n)$** → The heuristic (estimated) cost from node **n** to the goal node.
3. **$f(n) = g(n) + h(n)$** → The total estimated cost of the path through node **n** .

The algorithm selects the node with the lowest **$f(n)$** value, which balances actual cost (**$g(n)$**) and estimated future cost (**$h(n)$**).

A-Star Algorithm

Algorithm Steps

1. Initialize:

- Place the start node in the open list.
- Set $g(\text{start}) = 0$ and $f(\text{start}) = h(\text{start})$.
- Keep a **closed list** (visited nodes).

2. Processing Nodes:

- Pick the node with the **lowest $f(n) = g(n) + h(n)$** from the open list.
- If this node is the goal, the path is found.
- Otherwise, move it to the closed list.

A-Star Algorithm

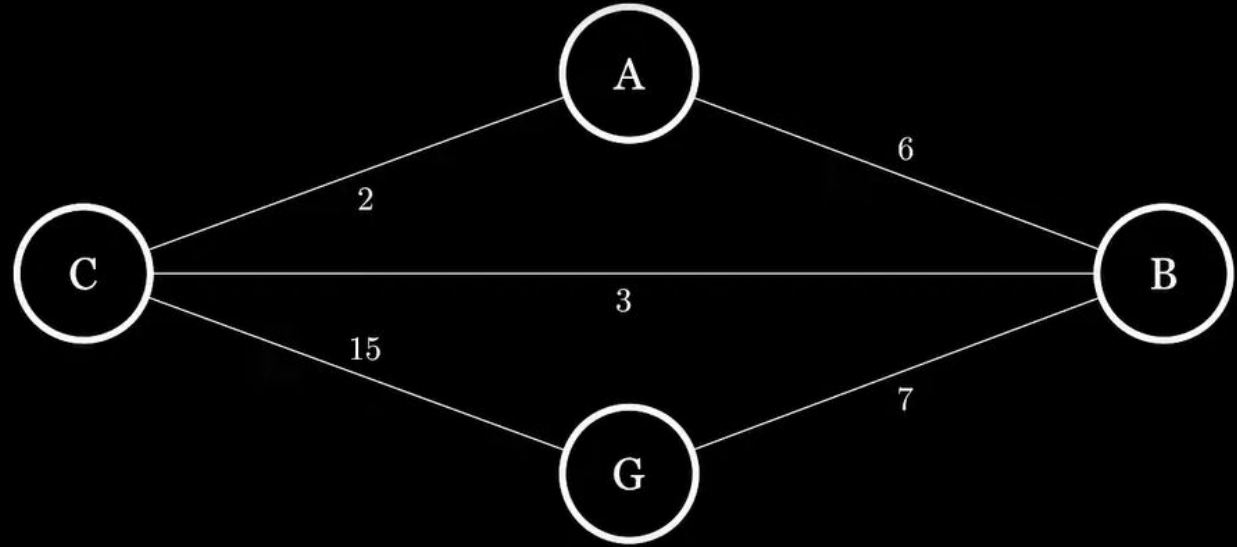
4. Expand Neighbors:

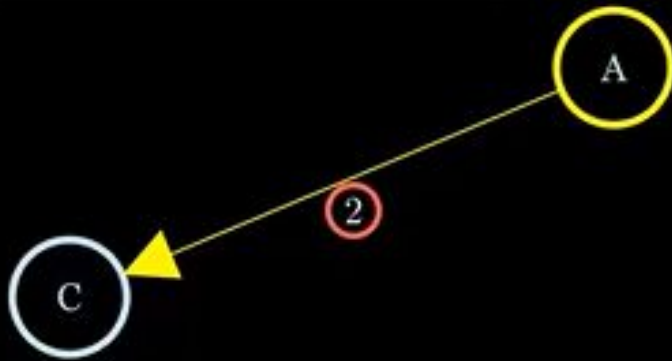
- Calculate new $g(n)$ for each neighbor.
- If a better path is found, update the values.
- Add unvisited neighbors to the open list.

5.Repeat until the goal is reached or no more nodes remain.

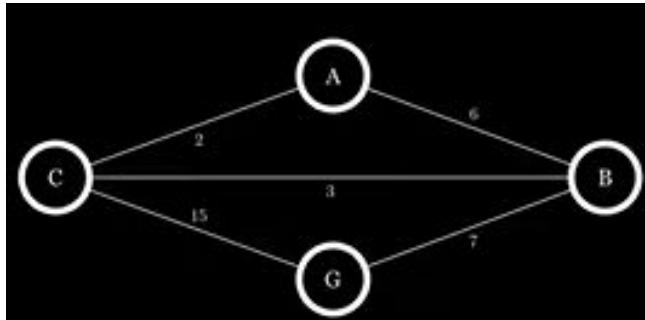
Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0



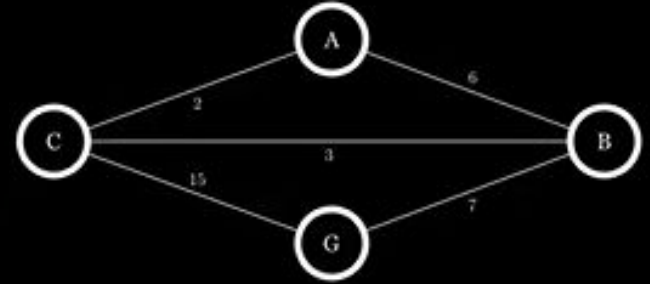
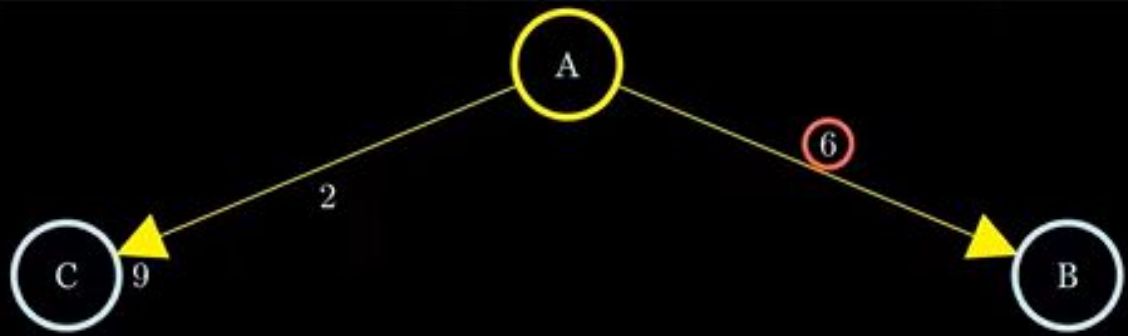


$$\text{Cost} = 2 + 7 = 9$$



Heuristic Table

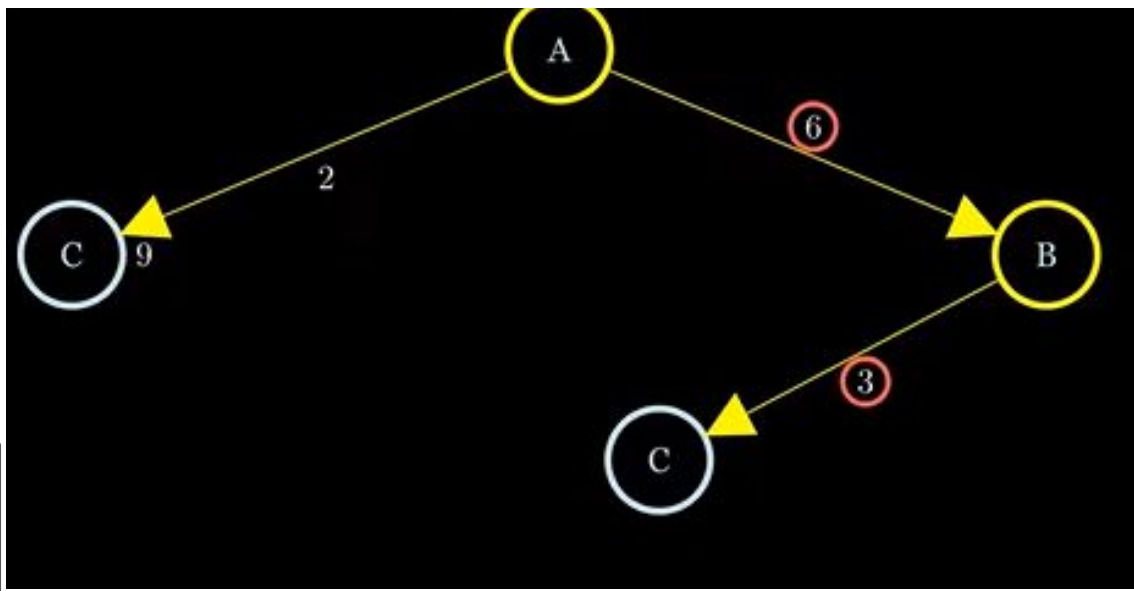
Node	Heuristic
A	10
B	2
C	7
G	0



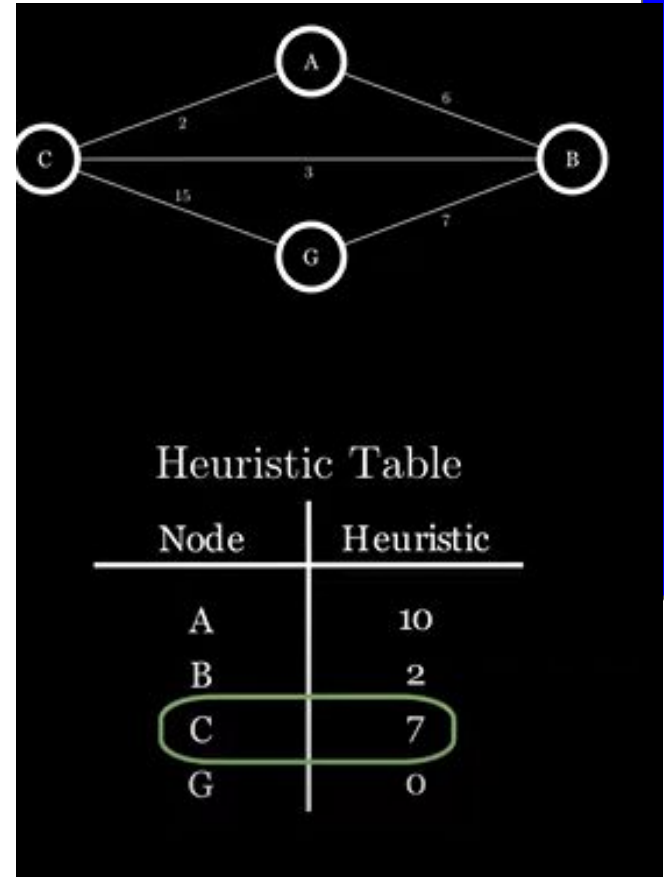
$$\text{Cost} = 6 + 2 = 8$$

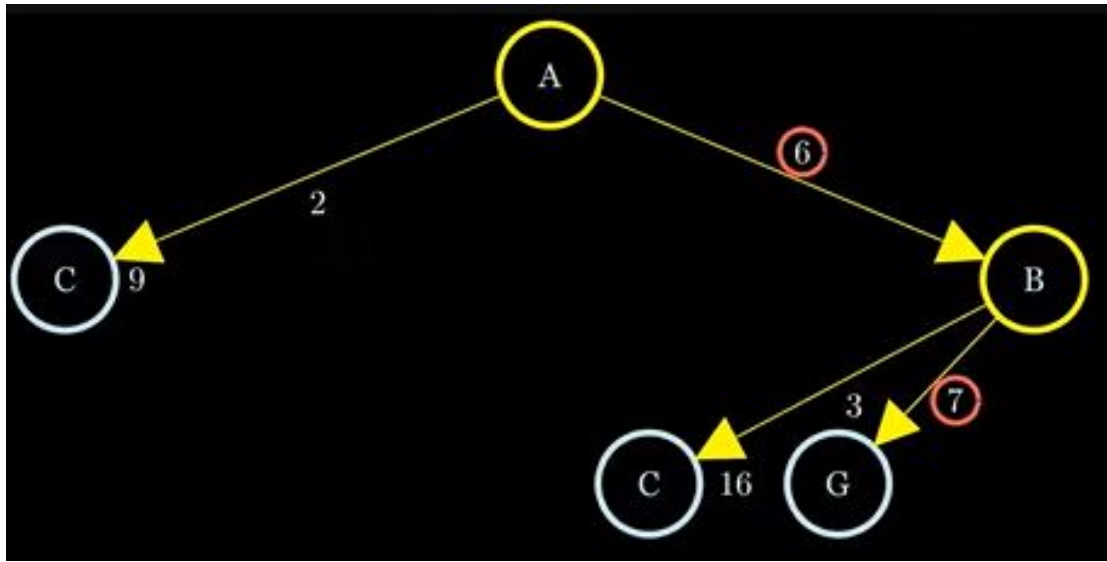
Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

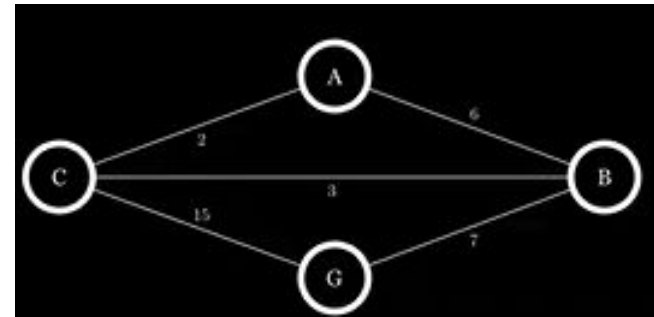


$$\text{Cost} = 6 + 3 + 7 = 16$$



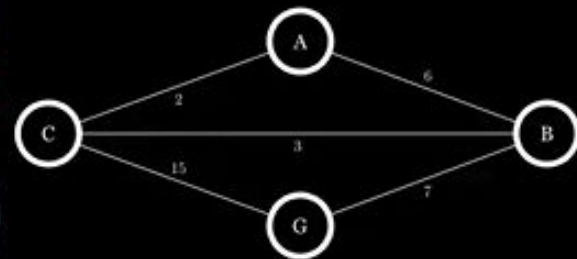
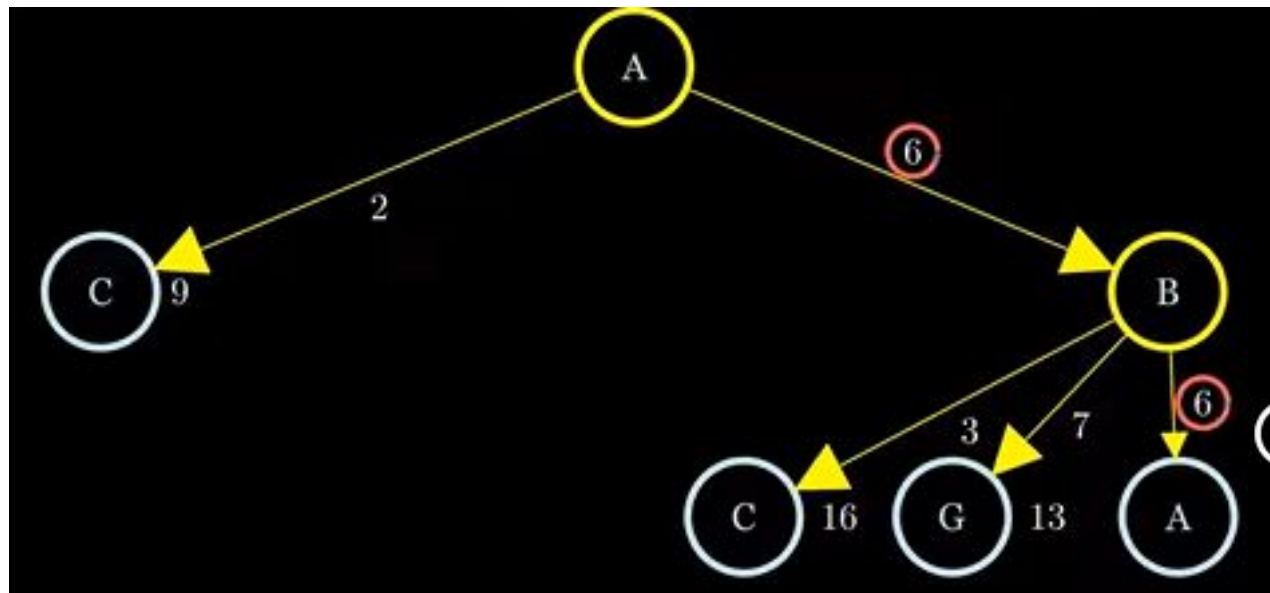


$$\text{Cost} = 6 + 7 + 0 = 13$$



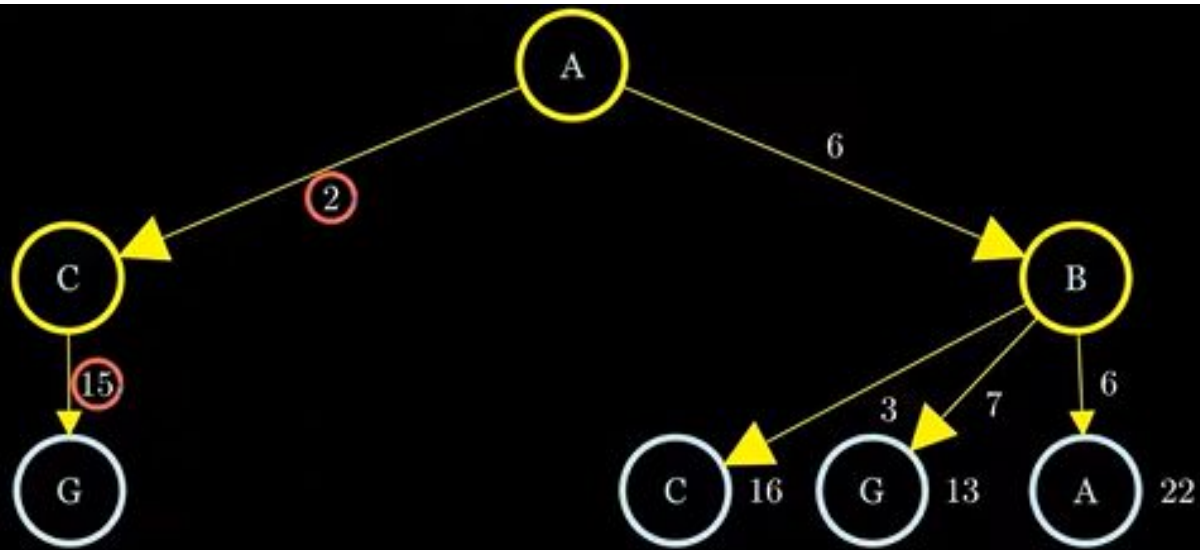
Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

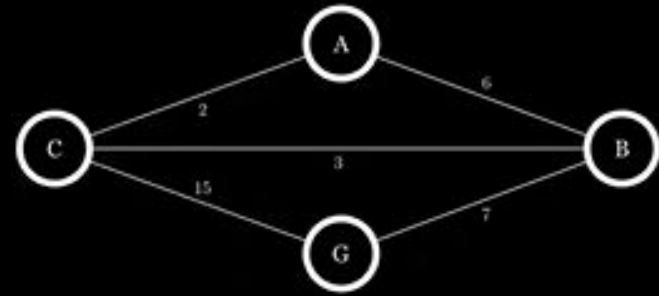


$$\text{Cost} = 6 + 6 + 10 = 22$$

Heuristic Table	
Node	Heuristic
A	10
B	2
C	7
G	0

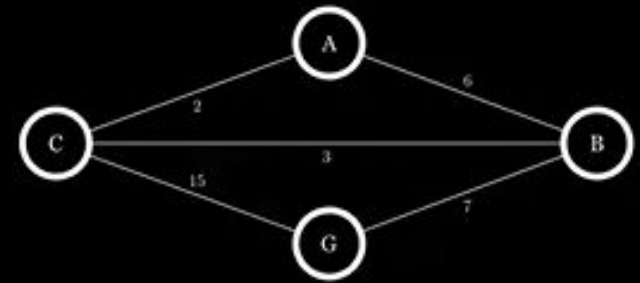
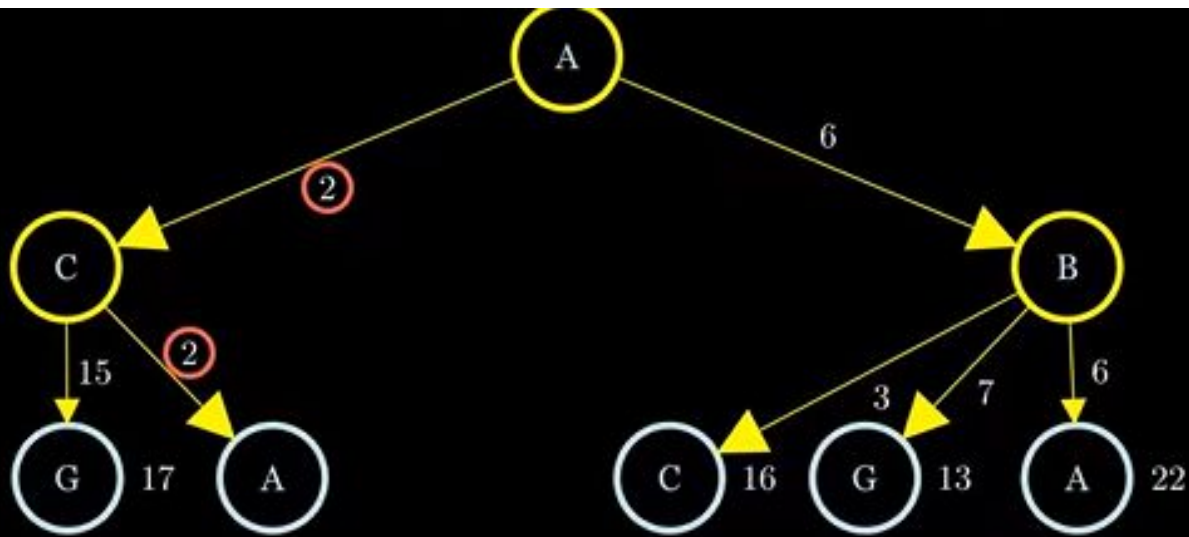


$$\text{Cost} = 2 + 15 + 0 = 17$$



Heuristic Table

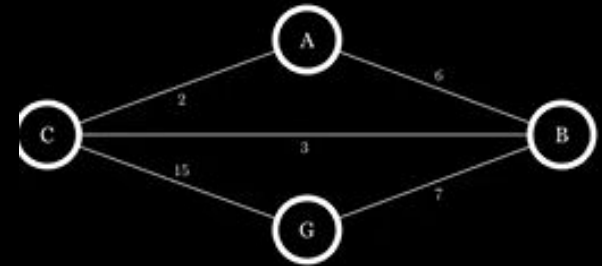
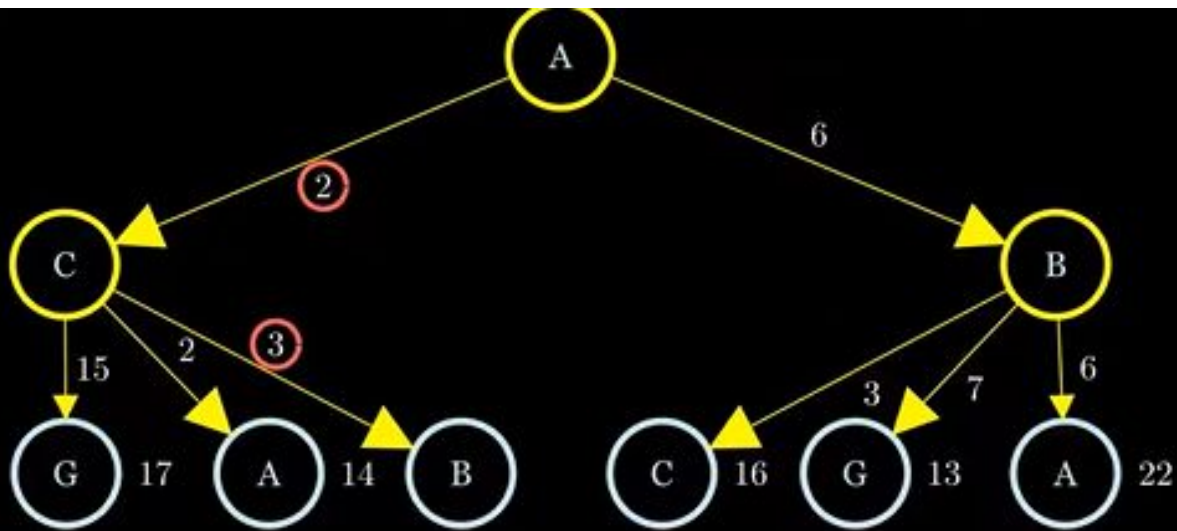
Node	Heuristic
A	10
B	2
C	7
G	0



Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

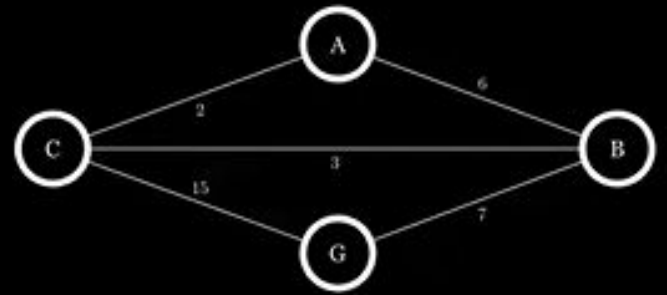
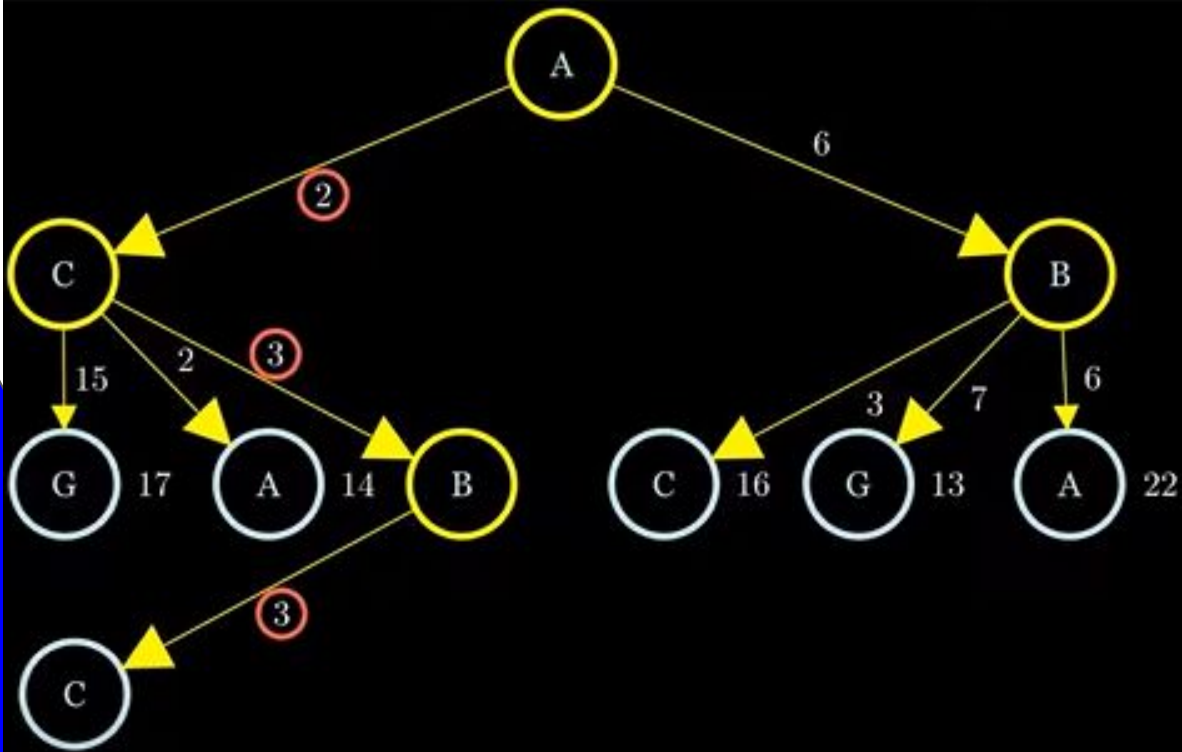
$$\text{Cost} = 2 + 2 + 10 = 14$$



Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

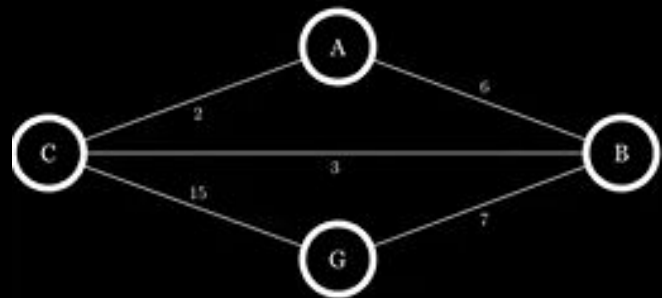
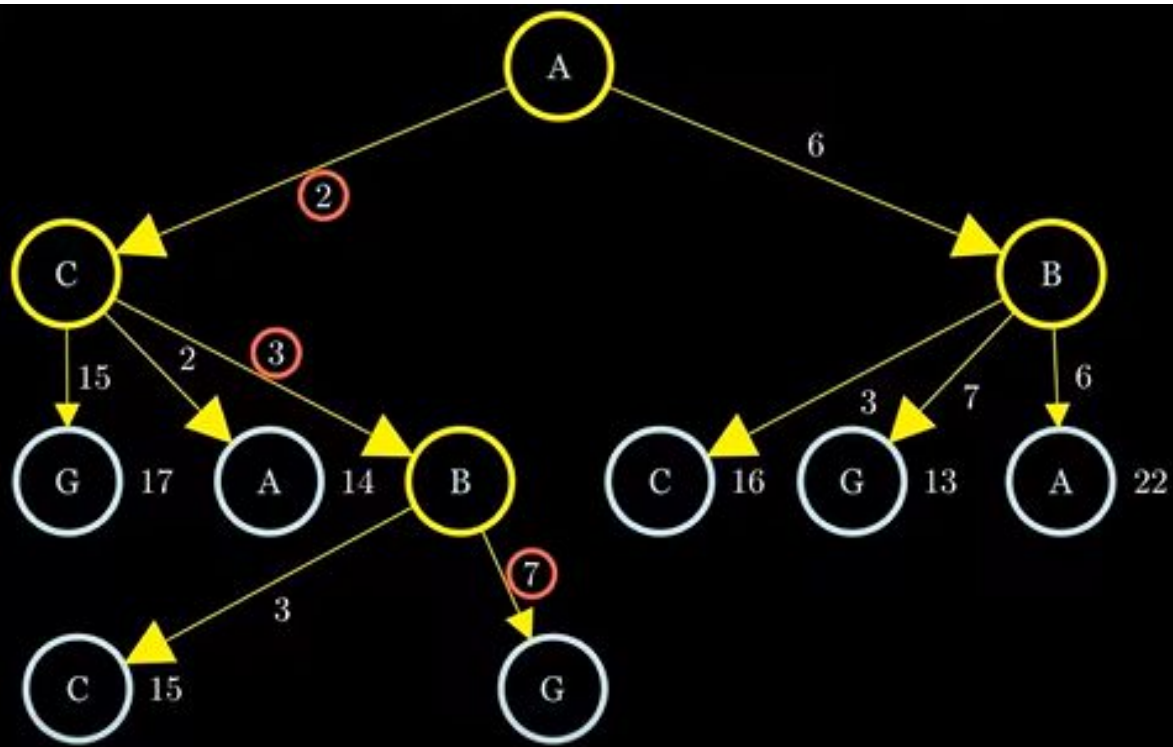
$$\text{Cost} = 2+3+2 = 7$$



Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

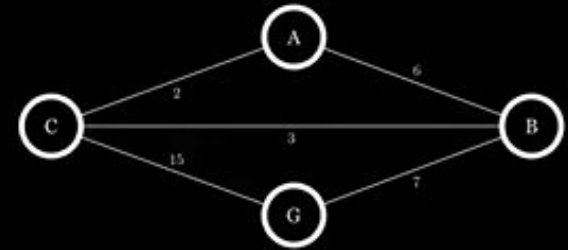
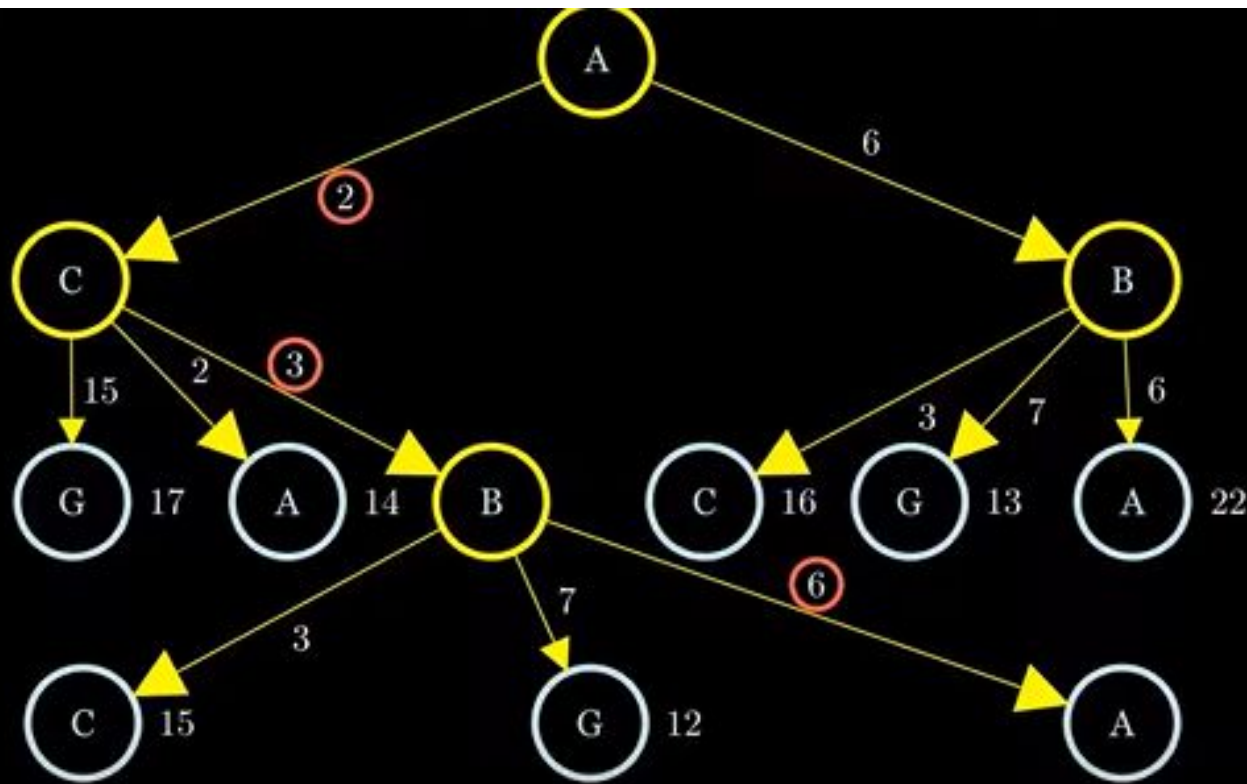
$$\text{Cost} = 2+3+3+7=15$$



Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

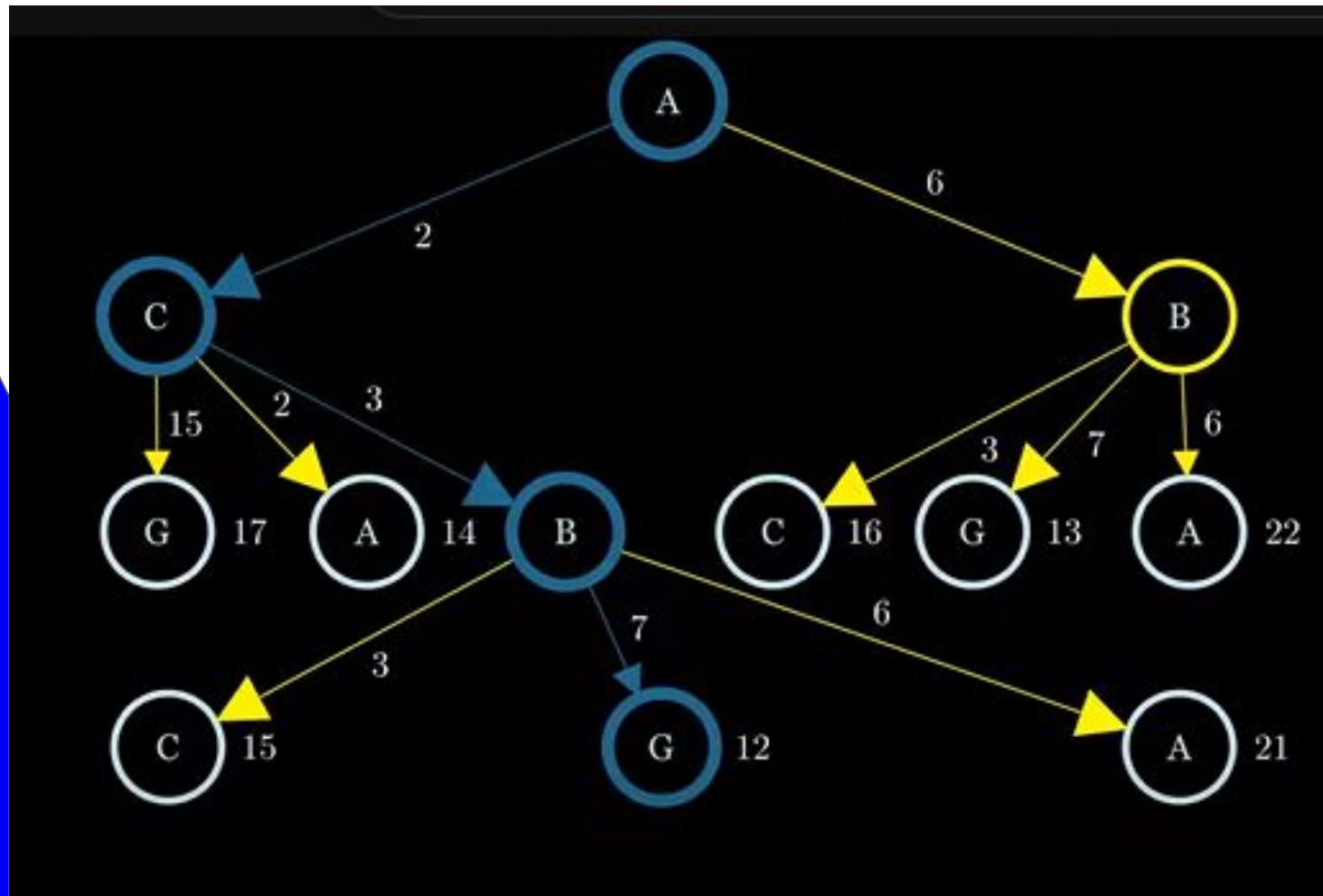
$$\text{Cost} = 2 + 3 + 7 + 0 = 12$$



Heuristic Table

Node	Heuristic
A	10
B	2
C	7
G	0

$$\text{Cost} = 2 + 3 + 6 + 10 = 21$$



D* Algorithm (Dynamic A*)

- The D (Dynamic A-Star) Algorithm* is an optimized path-planning algorithm used for dynamic environments where obstacles or costs may change over time.
- It is an extension of the A algorithm* that allows the shortest path to be recomputed efficiently when the environment changes.
- D* is widely used in robotics, autonomous vehicles, and AI for navigation in unpredictable environments.

D Algorithm (Dynamic A)**

Characteristics of D Algorithm*

- **Handles dynamic changes** in obstacles and terrain costs.
- **More efficient than re-running A*** when the environment updates.
- **Uses incremental search** to modify only affected areas of the path.
- **Commonly used in real-time applications** like robotics and self-driving cars.

D Algorithm (Dynamic A)**

*Why Use D Instead of A***

Feature	A* Algorithm	D* Algorithm
Environment	Static	Dynamic
Path Recalculation	Must rerun from scratch	Incrementally updates path
Efficiency	Slower in dynamic scenarios	Faster in changing environments
Best Used In	Games, GPS, static maps	Robotics, real-world navigation

D Algorithm (Dynamic A)**

Types of D Algorithms*

There are different versions of D* based on efficiency improvements:

1. **Classic D*** – The original version.
2. *D Lite** – A simplified and faster version, commonly used in modern applications.
3. **Field D*** – Uses continuous cost maps for smooth paths (used in terrain navigation).

Among these, *D Lite** is the most widely used version.

How D* Algorithm Works

1. Initial Path Calculation (Similar to A)*

- Computes the shortest path from the **goal to the start** (backward search).
- Uses **$g(n)$** (actual cost) and **$h(n)$** (heuristic estimate) to guide the search.

2. Dynamic Updates

- When the environment changes (e.g., an obstacle appears), *D updates only affected nodes** instead of recalculating the entire path.
- Uses **state propagation** to efficiently re-compute the new path.

3. Path Re-Evaluation

- If the current path is **blocked or suboptimal**, the algorithm **backtracks and finds an alternative path**.
- This **incremental search** significantly reduces computational cost.